Gabriel Castro - 25203195-2
José Flávio Xavier – 25357647-2
Pedro Lucas Silva Raimundo - 253631032
Wesley Brene - 251848252

BANCO DE ALIMENTOS DE LONDRINA

LONDRINA
2026

Gabriel Castro - 25203195-2
José Flávio Xavier – 25357647-2
Pedro Lucas Silva Raimundo - 253631032
Wesley Brene - 251848252

BANCO DE ALIMENTOS DE LONDRINA

Atividade de Estudo Programada apresentada ao Curso de Engenharia de Software da Universidade Cesumar, referente às disciplinas de Banco de Dados, Sistemas Operacionais, Programação Orientada a Objetos e Engenharia de Requisitos.

LONDRINA
2026

RESUMO

Este trabalho apresenta o desenvolvimento de um MVP para o Banco de Alimentos de Londrina, entidade fictícia que recebe doações e as distribui a instituições parceiras. O objetivo é substituir o controle por planilhas por um sistema que garanta a rastreabilidade da origem, da validade e do destino dos alimentos. O projeto integra quatro disciplinas. Em Banco de Dados, foram modelados e implementados em MariaDB tabelas, restrições, triggers, procedures, views e consultas, com 21 cenários de erro bloqueados. Em Sistemas Operacionais, foi desenvolvido um simulador em Java que usa mutex e semáforos para evitar condições de corrida e compara os algoritmos FIFO e LRU em um cache. Em Programação Orientada a Objetos, foi construído um sistema em Java com interface gráfica, CRUD, validações e 45 testes automatizados. Em Engenharia de Requisitos, foram especificados 24 requisitos funcionais, regras de negócio, casos de uso, rastreabilidade e gestão de mudanças. O resultado é um MVP funcional com persistência em memória, cujo banco de dados ainda não está ligado à aplicação, e uma especificação mais ampla que a implementação, o que fica registrado como próximo passo.

Palavras-chave: Banco de alimentos. Banco de dados. Sistemas operacionais. Programação orientada a objetos. Engenharia de requisitos.

SUMÁRIO

1 INTRODUÇÃO..... 5
2 DESENVOLVIMENTO..... 6
2.1 BANCO DE DADOS..... 6
2.1.1 Visão geral..... 6
2.1.2 Levantamento de requisitos de dados..... 6
2.1.3 Modelo conceitual..... 8
2.1.4 Modelo lógico..... 9
2.1.5 Normalização até a 3FN..... 10
2.1.6 Modelo físico e scripts DDL..... 11
2.1.7 Triggers, funções, procedures e views..... 12
2.1.8 Dados de teste (DML)..... 14
2.1.9 Consultas SQL de negócio..... 14
2.1.10 Testes de integridade..... 18
2.1.11 Segurança e backup (esboço)..... 24
2.1.12 Limitações..... 24
2.2 SISTEMAS OPERACIONAIS..... 25
2.2.1 Objetivo..... 25
2.2.2 Metodologia..... 25
2.2.3 Algoritmos de substituição de páginas..... 26
2.2.4 Mutex e semáforos..... 28
2.2.5 Prevenção de condições de corrida e deadlock..... 29
2.2.6 Análise dos dados..... 30
2.2.7 Conclusão da disciplina..... 30
2.3 PROGRAMAÇÃO ORIENTADA A OBJETOS..... 31
2.3.1 Proposta, usuários e glossário..... 31
2.3.2 Requisitos funcionais e não funcionais..... 31
2.3.3 Regras de negócio..... 32
2.3.4 Casos de uso e critérios de aceitação..... 33
2.3.5 Visão geral da 2ª entrega..... 33
2.3.6 Diagrama de classes refatorado..... 34
2.3.7 Implementação das classes e associações..... 36
2.3.8 Herança, polimorfismo, sobrecarga, interfaces e classes abstratas..... 37
2.3.9 Coleções, exceções e validações..... 37
2.3.10 Interface gráfica..... 38
2.3.11 Fluxo CRUD completo..... 41
2.3.12 Testes..... 41
2.3.13 Documentação e código-fonte..... 42
2.4 ENGENHARIA DE REQUISITOS..... 43
2.4.1 Contexto, objetivo e escopo..... 43
2.4.2 Partes interessadas e levantamento..... 43
2.4.3 Requisitos funcionais e priorização (MoSCoW)..... 44
2.4.4 Requisitos não funcionais..... 45
2.4.5 Regras de negócio..... 45
2.4.6 Casos de uso..... 46
2.4.7 Rastreabilidade..... 49

2.4.8 Divergências entre a ERS e a implementação.....52
2.4.9 Protótipo e validação dos requisitos.....53
2.4.10 Gestão de mudanças.....54
3 CONSIDERAÇÕES FINAIS.....56
REFERÊNCIAS.....57
BIBLIOGRAFIA CONSULTADA.....58

1 INTRODUÇÃO

O Banco de Alimentos de Londrina é uma entidade fictícia, com operação realista, que recebe doações de mercados, atacadistas e produtores locais e as distribui a instituições parceiras, como creches, casas de acolhimento e cozinhas solidárias. Hoje o controle é feito por planilhas e formulários manuais, o que dificulta a rastreabilidade (origem, validade e destino dos alimentos), o planejamento logístico, a análise de indicadores e a prestação de contas (UNICESUMAR, 2026, p. 1).

O desafio proposto na Atividade de Estudos Programados (AEP) é desenvolver um MVP com base de dados estruturada e gestão do projeto formalizada, para digitalizar o processo de doações, estoque e distribuição, com foco em integridade, performance e governança dos dados (UNICESUMAR, 2026, p. 1).

Este trabalho reúne as entregas de quatro disciplinas do 4º semestre de Engenharia de Software. A seção 2.1 trata de Banco de Dados (modelagem, scripts, objetos e consultas); a seção 2.2, de Sistemas Operacionais (sincronização de threads e substituição de cache); a seção 2.3, de Programação Orientada a Objetos (sistema em Java com interface gráfica e testes); e a seção 2.4, de Engenharia de Requisitos (ERS, casos de uso, rastreabilidade e gestão de mudanças).

O código-fonte, os scripts SQL, o simulador e os documentos estão no repositório do projeto: https://github.com/JoseFlavioXavier/AEP_ENTREGA_02.

2 DESENVOLVIMENTO

2.1 BANCO DE DADOS

2.1.1 Visão geral

O banco de dados guarda doadores, instituições, alimentos, doações, lotes, distribuições e perdas do Banco de Alimentos de Londrina. O foco do enunciado é a integridade, a rastreabilidade (origem, validade e destino de cada alimento) e os indicadores de estoque (UNICESUMAR, 2026, p. 1 e 6). O SGBD usado foi o MariaDB 10.11 (versão mínima 10.6, por causa da função JSON_TABLE usada nas procedures).

Os scripts ficam na pasta banco do projeto e são executados em ordem pelo arquivo 00_executar_tudo.sql. Cada arquivo cobre um item da 2ª entrega: DDL e restrições (01), triggers (02), funções e procedures (03), views (04), dados de teste (05), consultas de negócio (06) e testes de erro (07). O script apaga e recria o banco banco_alimentos, então pode ser repetido do zero.

Quadro 1 – Arquivos de script e conteúdo

Arquivo	Conteúdo
`01_ddl.sql`	Banco, 9 tabelas, chaves, restrições CHECK/UNIQUE/FK, 10 índices e carga das categorias
`02_triggers.sql`	12 triggers: saldo do lote, bloqueio de saída inválida e proteção do histórico
`03_funcoes_procedures.sql`	2 funções (dias para vencer e situação do lote) e 3 procedures transacionais
`04_views.sql`	6 views: estoque por validade, doações por período, distribuições por trimestre, indicadores por categoria, rastreabilidade e conferência de saldo
`05_dados_teste.sql`	Dados mínimos coerentes para demonstrar as funcionalidades
`06_consultas.sql`	12 consultas SQL de negócio
`07_testes_erros.sql`	21 cenários inválidos que devem ser bloqueados e conferência final

Fonte: Elaborado para este trabalho (2026).

2.1.2 Levantamento de requisitos de dados

O Quadro 2 é o glossário de dados: cada entidade, o que ela representa e a chave que a identifica. O Quadro 3 reúne as regras de negócio que se aplicam aos dados e o mecanismo do banco que garante cada uma.

Quadro 2 – Glossário de dados e entidades

Entidade	Descrição	Chave primária
`doador`	Pessoa física ou jurídica que doa alimentos. Tem tipo, documento (CPF ou CNPJ, só dígitos) e contato.	`id_doador`
`instituicao`	Creche, casa de acolhimento ou cozinha solidária que recebe alimentos. Tem documento, endereço e capacidade de atendimento.	`id_instituicao`
`categoria`	Classificação do alimento (Grãos, Enlatados, Laticínios, Hortifrúti, Proteínas, Bebidas, Outros).	`id_categoria`
`alimento`	Tipo de alimento, com nome único, categoria e unidade de medida (kg, g, l, ml ou un).	`id_alimento`
`doacao`	Entrada de alimentos feita por um doador em uma data.	`id_doacao`
`lote`	Parte de uma doação para um alimento, com quantidade inicial, saldo e validade. É a unidade de controle do estoque.	`id_lote`
`distribuicao`	Saída de alimentos para uma instituição em uma data.	`id_distribuicao`
`item_distribuicao`	Quantidade retirada de um lote em uma distribuição.	`id_item_distribuicao`
`perda`	Baixa de parte de um lote, com quantidade, motivo e data.	`id_perda`

Fonte: Elaborado para este trabalho (2026).

Quadro 3 – Regras de negócio dos dados

Código	Regra	Como o banco garante
RD01	Nome, contato e documento do doador são obrigatórios; o documento é único e tem 11 dígitos (pessoa física) ou 14 (pessoa jurídica).	CHECK e UNIQUE em doador
RD02	Instituição tem documento único de 14 dígitos e capacidade maior que zero.	CHECK e UNIQUE em instituicao
RD03	O nome do alimento é único, sem diferenciar maiúsculas e minúsculas, e toda categoria deve existir.	UNIQUE (collation unicode_ci) e FK
RD04	Uma doação ou distribuição precisa ter pelo menos um item.	Procedures sp_registrar_doacao e sp_registrar_distribuicao
RD05	O saldo do lote nasce igual à quantidade inicial e nunca fica negativo nem maior que a quantidade inicial.	Trigger trg_lote_bi e CHECK ck_lote_saldo
RD06	Não se retira nem se perde mais do que o saldo do lote.	Triggers trg_item_distribuicao_bi e trg_perda_bi
RD07	Lote com validade anterior à data da doação ou da saída é recusado.	Triggers trg_lote_bi e trg_item_distribuicao_bi
RD08	O saldo é atualizado automaticamente a cada saída ou perda.	Triggers trg_item_distribuicao_ai e trg_perda_ai
RD09	Perda exige quantidade positiva e motivo informado.	CHECK em perda

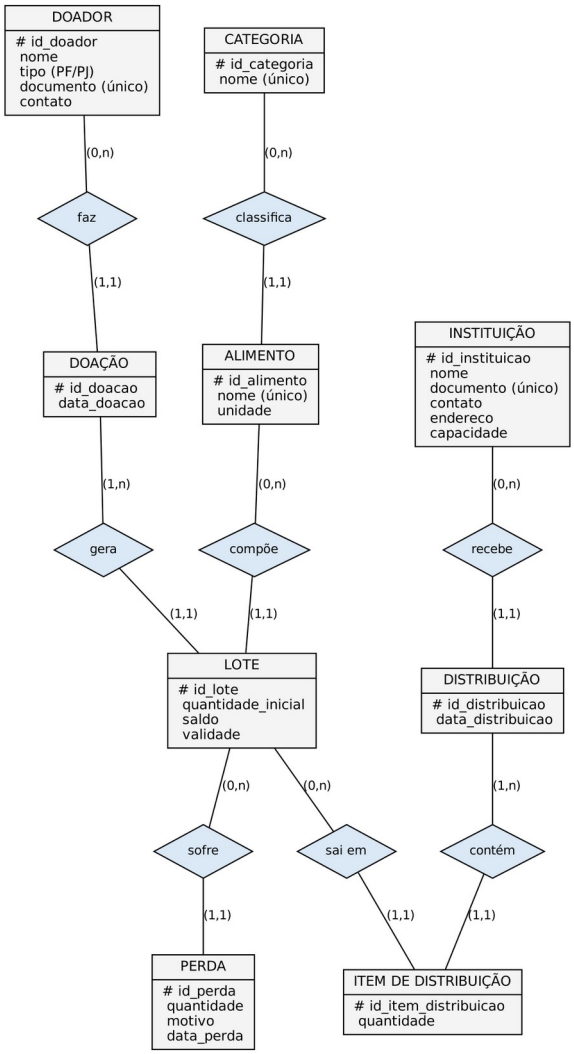
Código	Regra	Como o banco garante
RD10	A unidade de medida do alimento é kg, g, l, ml ou un e não pode mudar depois que o alimento tem lotes.	CHECK ck_alimento_unidade e trigger trg_alimento_bu
RD11	O histórico não pode ser apagado ou alterado: lotes, itens de distribuição e perdas ficam protegidos, e doador, instituição, alimento e doação usados não podem ser excluídos.	Triggers de UPDATE/DELETE e FK com ON DELETE RESTRICT

Fonte: Elaborado para este trabalho (2026).

2.1.3 Modelo conceitual

O modelo conceitual da 1ª entrega foi refeito. A versão anterior usava uma entidade central chamada “Banco de alimentos”, que misturava dados de alimento, lote, saldo e doação, e não separava a perda. Na versão atual cada conceito do glossário é uma entidade própria. A Figura 1 mostra o diagrama com as cardinalidades (mínima, máxima) de cada lado do relacionamento.

Figura 1 – Modelo conceitual (DER)



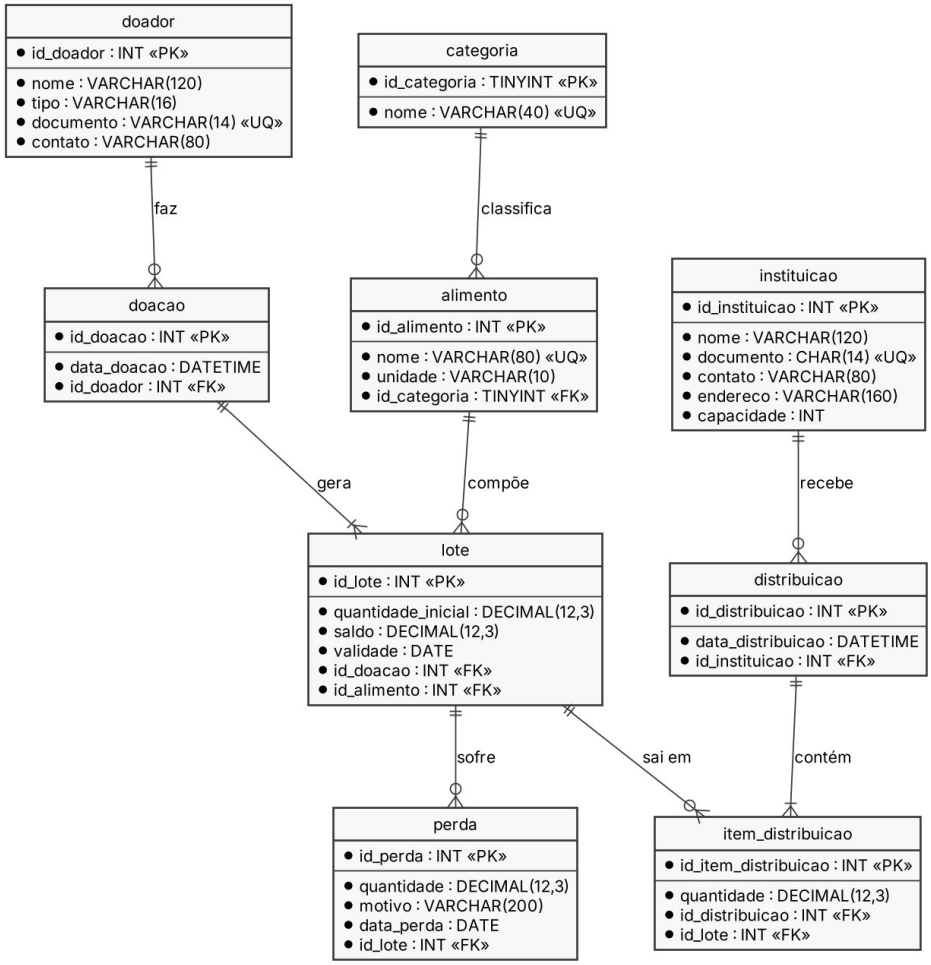
Fonte: Elaborado para este trabalho (2026).

As principais decisões foram: o lote é a entidade que liga a doação ao alimento e guarda validade e saldo, porque uma doação pode trazer vários alimentos com validades diferentes; a saída de um lote é registrada em item_distribuicao, o que permite uma distribuição com vários lotes; e a perda é uma entidade separada, para que os indicadores de perda e a rastreabilidade não dependam do saldo.

2.1.4 Modelo lógico

A Figura 2 mostra o modelo relacional gerado a partir do modelo conceitual, com atributos, tipos, chaves primárias (PK), estrangeiras (FK), chaves únicas (UQ) e cardinalidades. Todos os relacionamentos 1:N viraram uma chave estrangeira no lado N.

Figura 2 – Modelo lógico (relacional)



Fonte: Elaborado para este trabalho (2026).

A cardinalidade “1..*” entre doação e lote (e entre distribuição e item) é garantida pelas procedures de registro. O banco, sozinho, aceitaria uma linha de doacao sem lote, como acontece por instantes na carga de dados de teste, que insere a doação e depois os lotes.

2.1.5 Normalização até a 3FN

Quadro 4 – Decisões de normalização

Forma normal	Decisão aplicada
1FN	Todos os atributos são atômicos. Os vários alimentos de uma doação não ficam em uma coluna repetida: cada um é uma linha de lote. Da mesma forma, os vários lotes de uma distribuição são linhas de item_distribuicao.
2FN	Todas as tabelas têm chave primária simples (id autoincrementado), então não existe dependência parcial de parte de uma chave composta.
3FN	Não há dependência entre atributos que não são chave. A categoria foi separada do alimento (tabela categoria), e doador e instituição são tabelas distintas, cada uma com seus próprios dados. O nome do doador e do alimento não se repete em lote ou doação: eles são lidos por junção.

Forma normal	Decisão aplicada
Exceção controlada	lote.saldo é um valor derivado (quantidade inicial menos saídas e perdas). Ele foi mantido de propósito para a consulta de estoque não somar o histórico a cada leitura. A consistência é garantida pelos triggers e conferida pela view vw_conferencia_saldo, que recalcula o saldo e compara com o valor gravado.

Fonte: Elaborado para este trabalho (2026).

2.1.6 Modelo físico e scripts DDL

O modelo físico está no arquivo 01_ddl.sql. Todas as tabelas usam o mecanismo InnoDB, que tem transações e chaves estrangeiras, e o conjunto de caracteres utf8mb4. As quantidades usam DECIMAL(12,3), porque o estoque tem kg e litros fracionados e não pode sofrer erro de arredondamento de ponto flutuante. As chaves estrangeiras usam ON UPDATE CASCADE e ON DELETE RESTRICT, para impedir a exclusão de um registro que já tem histórico.

Quadro 5 – Restrições CHECK, UNIQUE e FK

Tabela	Restrição	Efeito

doador	`uq_doador_documento`
	`ck_doador_nome, ck_doador_contato`
	`ck_doador_tipo`
	`ck_doador_documento`
		Documento único
		Nome e contato não vazios
		Tipo PESSOA_FISICA ou PESSOA_JURIDICA
		Só dígitos, 11 (PF) ou 14 (PJ)

instituicao	`uq_instituicao_documento`
	`ck_instituicao_documento`
	`ck_instituicao_capacidade`
	`ck_instituicao_nome, _contato, _endereco`
		Documento único
		14 dígitos
		Capacidade maior que zero
		Campos não vazios

categoria	`uq_categoria_nome`	Nome único

alimento	`uq_alimento_nome`
	`fk_alimento_categoria`
	`ck_alimento_nome`
	`ck_alimento_unidade`
		Nome único sem diferenciar maiúsculas
		Categoria existente
		Nome não vazio
		Unidade kg, g, l, ml ou un

doacao	`fk_doacao_doador`	Doador existente

lote	`fk_lote_doacao, fk_lote_alimento`
	`ck_lote_quantidade_inicial`
	`ck_lote_saldo`
		Doação e alimento existentes
		Quantidade inicial maior que zero
		Saldo entre 0 e a quantidade inicial

distribuicao	`fk_distribuicao_instituicao`	Instituição existente

item_distribuicao	`fk_item_distribuicao_distribuicao`
	`fk_item_distribuicao_lote`
	`ck_item_distribuicao_quantidade`
		Distribuição e lote existentes
		Quantidade maior que zero

perda	`fk_perda_lote`
	`ck_perda_quantidade`
	`ck_perda_motivo`
		Lote existente
		Quantidade maior que zero
		Motivo não vazio

Fonte: Elaborado para este trabalho (2026).

Quadro 6 – Índices criados

Índice	Colunas	Consulta que atende
idx_doacao_data	doacao (data_doacao)	Doações por período
idx_doacao_doador_data	doacao (id_doador, data_doacao)	Ranking e histórico por doador
idx_lote_validade	lote (validade)	Estoque por validade e alertas de vencimento
idx_lote_alimento_validade	lote (id_alimento, validade)	Lotes de um alimento ordenados por validade
idx_lote_doacao	lote (id_doacao)	Lotes de uma doação
idx_distribuicao_data	distribuicao (data_distribuicao)	Distribuições por período
idx_distribuicao_instituicao_data	distribuicao (id_instituicao, data_distribuicao)	Distribuição por instituição no trimestre
idx_item_distribuicao_lote	item_distribuicao (id_lote)	Rastreabilidade e soma de saídas por lote
idx_perda_lote	perda (id_lote)	Rastreabilidade e soma de perdas por lote
idx_perda_data	perda (data_perda)	Perdas por período

Fonte: Elaborado para este trabalho (2026).

2.1.7 Triggers, funções, procedures e views

Os triggers mantêm o saldo e protegem o histórico. A regra de estoque insuficiente e de lote vencido fica no banco, e não só no programa, assim qualquer sistema que grave nele é obrigado a respeitá-la. Os triggers de saída e de perda leem o lote com FOR UPDATE, o que trava a linha enquanto a transação roda e evita que duas saídas simultâneas usem o mesmo saldo.

Quadro 7 – Triggers

Trigger	Evento	Função
trg_alimento_bu	BEFORE UPDATE em alimento	Bloqueia a troca da unidade de medida quando o alimento já tem lotes
trg_lote_bi	BEFORE INSERT em lote	Recusa validade anterior à data da doação e define saldo igual à quantidade inicial
trg_lote_bu	BEFORE UPDATE em lote	Bloqueia alteração de doação, alimento, quantidade inicial e validade; só o saldo muda
trg_lote_bd	BEFORE DELETE em lote	Bloqueia a exclusão de lotes
trg_item_distribuicao_bi	BEFORE INSERT em item_distribuicao	Recusa lote vencido na data da saída e quantidade maior que o saldo

Trigger	Evento	Função
`trg_item_distribuicao_ai`	AFTER INSERT em item_distribuicao	Desconta a quantidade do saldo do lote
`trg_item_distribuicao_bu / _bd`	BEFORE UPDATE / DELETE	Bloqueiam alteração e exclusão de itens de distribuição
`trg_perda_bi`	BEFORE INSERT em perda	Recusa perda maior que o saldo
`trg_perda_ai`	AFTER INSERT em perda	Desconta a perda do saldo do lote
`trg_perda_bu / _bd`	BEFORE UPDATE / DELETE	Bloqueiam alteração e exclusão de perdas

Fonte: Elaborado para este trabalho (2026).

Quadro 8 – Funções e procedures

Objeto	O que faz
`fn_dias_para_vencer(validade)`	Devolve os dias entre hoje e a validade (negativo quando vencido)
`fn_situacao_lote(saldo, validade)`	Devolve Esgotado, Vencido, Vence em até 7 dias ou OK
`sp_registrar_doacao(doador, itens, OUT id)`	Cria a doação e todos os lotes em uma transação. Recebe os itens em JSON. Recusa lista vazia e faz ROLLBACK em qualquer erro
`sp_registrar_distribuicao(instituicao, itens, OUT id)`	Cria a distribuição e os itens em uma transação. Se um item falha, nenhum saldo muda
`sp_registrar_perda(lote, quantidade, motivo)`	Registra a perda em uma transação, com o saldo conferido pelo trigger

Fonte: Elaborado para este trabalho (2026).

Quadro 9 – Views

View	Conteúdo
`vw_estoque_validade`	Lotes com alimento, categoria, doador de origem, saldo, validade, dias para vencer e situação
`vw_doacoes_periodo`	Doações, lotes e quantidade total por mês e unidade
`vw_distribuicoes_trimestre`	Distribuições e quantidade por instituição, trimestre e unidade
`vw_indicadores_categoria`	Doado, distribuído, perdido, saldo, giro e perda percentual por categoria e unidade
`vw_rastreabilidade_lote`	Histórico de cada lote: origem, destinos das distribuições e perdas
`vw_conferencia_saldo`	Compara o saldo gravado com o saldo recalculado (OK ou DIVERGENTE)

Fonte: Elaborado para este trabalho (2026).

As quantidades de unidades diferentes (kg, l e un) nunca são somadas entre si: todas as views e consultas agrupam também por unidade.

2.1.8 Dados de teste (DML)

O arquivo 05_dados_teste.sql carrega 5 doadores (4 pessoas jurídicas e 1 pessoa física), 4 instituições, 10 alimentos nas categorias do banco, 6 doações com 13 lotes, 5 distribuições e 3 perdas. As quatro primeiras doações e as três primeiras distribuições são inseridas diretamente; as outras duas doações, as duas últimas distribuições e uma perda passam pelas procedures, o que também testa as procedures. As validades são relativas ao dia da execução, então sempre há lotes vencidos (banana e tomate da cooperativa) e lotes que vencem em até 7 dias (leite e iogurte da padaria).

2.1.9 Consultas SQL de negócio

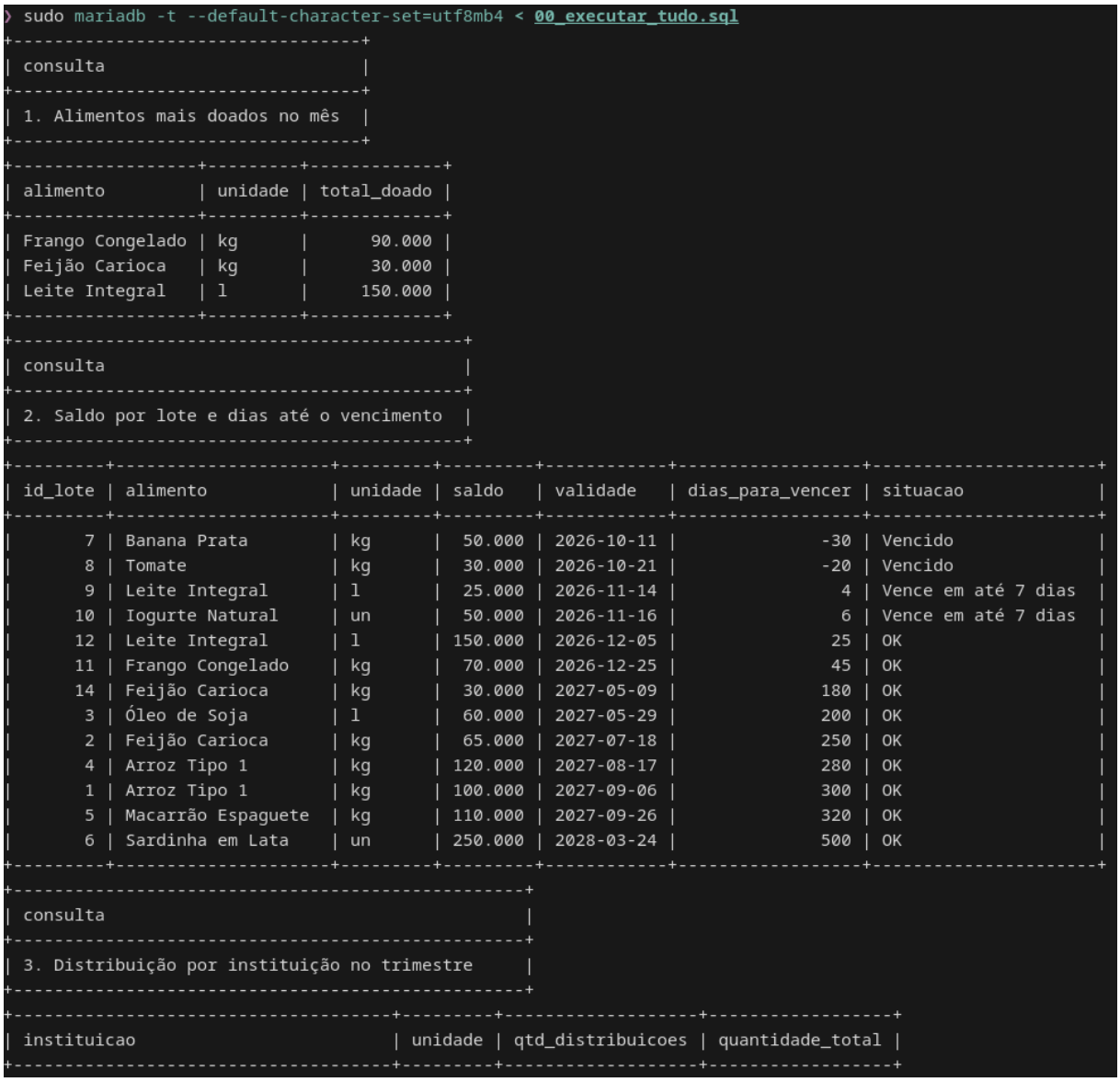
O arquivo 06_consultas.sql traz 12 consultas, que respondem às perguntas do gestor e incluem os três exemplos citados no enunciado (alimentos mais doados no mês, saldo por lote e dias até o vencimento, distribuição por instituição no trimestre) (UNICESUMAR, 2026, p. 6). As Figuras 3 a 6 mostram a execução completa no terminal, em capturas sequenciais que se sobrepõem um pouco.

Quadro 10 – Consultas de negócio

Nº	Pergunta de negócio	Objeto usado
1	Quais alimentos foram mais doados no mês?	doacao, lote, alimento
2	Qual o saldo de cada lote e quantos dias faltam para vencer?	vw_estoque_validade
3	Quanto cada instituição recebeu no trimestre?	vw_distribuicoes_trimestre
4	Quais lotes vencem em até 7 dias (alerta)?	vw_estoque_validade
5	Quais lotes vencidos ainda têm saldo (perda a registrar)?	vw_estoque_validade
6	Qual o saldo atual por categoria (sem vencidos)?	vw_estoque_validade
7	Qual o giro e a perda por categoria?	vw_indicadores_categoria
8	Quanto foi doado por período?	vw_doacoes_periodo
9	Qual o ranking de doadores?	doador, doacao, lote
10	Quais instituições não receberam nada nos últimos 30 dias?	instituicao, distribuicao
11	Qual a origem, o destino e as perdas do leite (rastreabilidade)?	vw_rastreabilidade_lote
12	O saldo gravado em cada lote bate com o histórico?	vw_conferencia_saldo

Fonte: Elaborado para este trabalho (2026).

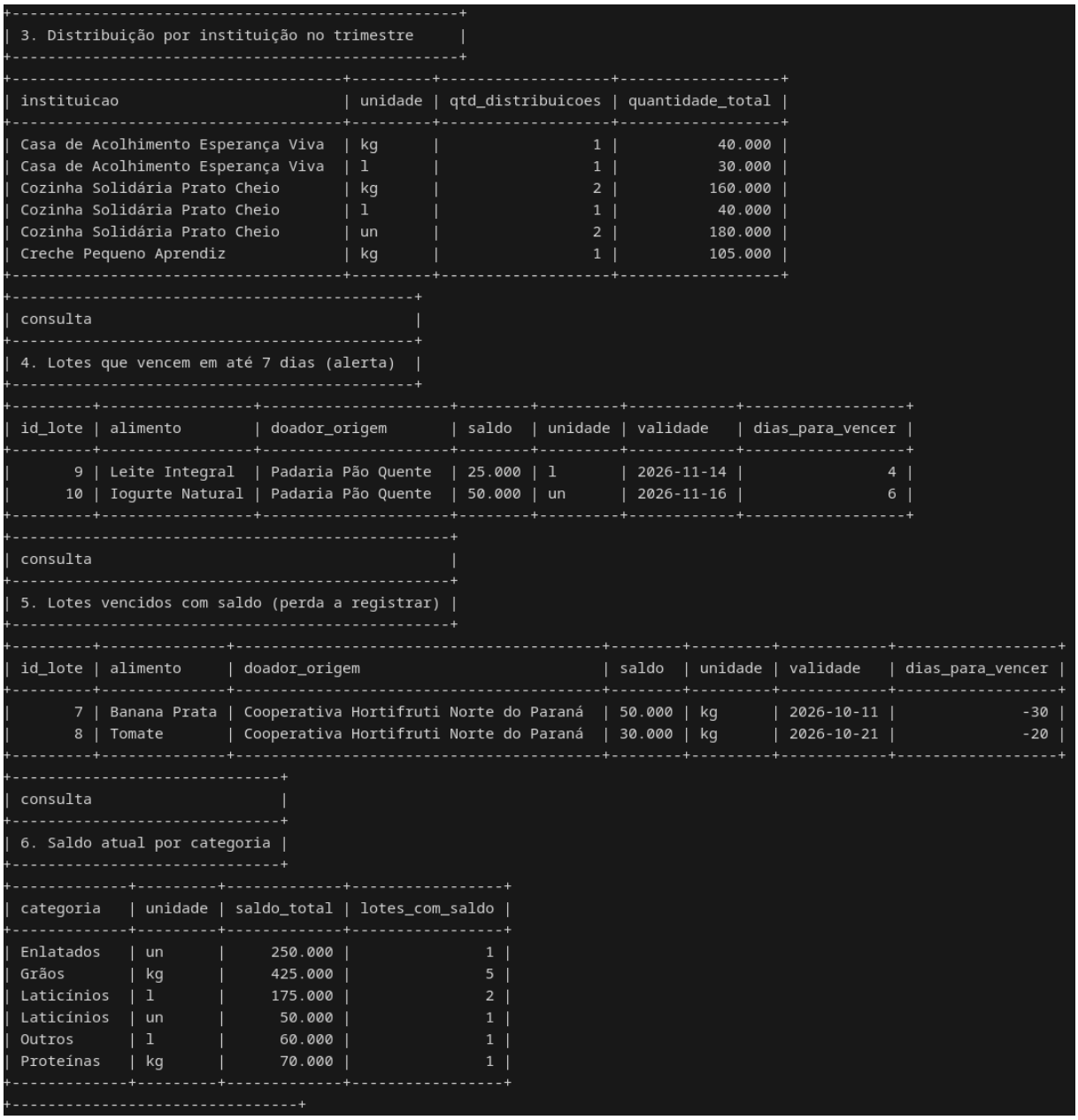
Figura 3 – Execução das consultas 1 a 3



```
❯ sudo mariadb -t --default-character-set=utf8mb4 < 00_executar_tudo.sql
+---------------------------------+
| consulta                        |
+---------------------------------+
| 1. Alimentos mais doados no mês  |
+---------------------------------+
+------------------+---------+-------------+
| alimento         | unidade | total_doado |
+------------------+---------+-------------+
| Frango Congelado | kg      |      90.000 |
| Feijão Carioca   | kg      |      30.000 |
| Leite Integral   | l       |     150.000 |
+------------------+---------+-------------+
+-----------------------------------------+
| consulta                                |
+-----------------------------------------+
| 2. Saldo por lote e dias até o vencimento  |
+-----------------------------------------+
+---------+---------------------+---------+---------+------------+------------------+----------------------+
| id_lote | alimento            | unidade | saldo   | validade   | dias_para_vencer | situacao             |
+---------+---------------------+---------+---------+------------+------------------+----------------------+
|       7 | Banana Prata        | kg      |  50.000 | 2026-10-11 |              -30 | Vencido              |
|       8 | Tomate              | kg      |  30.000 | 2026-10-21 |              -20 | Vencido              |
|       9 | Leite Integral      | l       |  25.000 | 2026-11-14 |                4 | Vence em até 7 dias  |
|      10 | Iogurte Natural     | un      |  50.000 | 2026-11-16 |                6 | Vence em até 7 dias  |
|      12 | Leite Integral      | l       | 150.000 | 2026-12-05 |               25 | OK                   |
|      11 | Frango Congelado    | kg      |  70.000 | 2026-12-25 |               45 | OK                   |
|      14 | Feijão Carioca      | kg      |  30.000 | 2027-05-09 |              180 | OK                   |
|       3 | Óleo de Soja        | l       |  60.000 | 2027-05-29 |              200 | OK                   |
|       2 | Feijão Carioca      | kg      |  65.000 | 2027-07-18 |              250 | OK                   |
|       4 | Arroz Tipo 1        | kg      | 120.000 | 2027-08-17 |              280 | OK                   |
|       1 | Arroz Tipo 1        | kg      | 100.000 | 2027-09-06 |              300 | OK                   |
|       5 | Macarrão Espaguete  | kg      | 110.000 | 2027-09-26 |              320 | OK                   |
|       6 | Sardinha em Lata    | un      | 250.000 | 2028-03-24 |              500 | OK                   |
+---------+---------------------+---------+---------+------------+------------------+----------------------+
+-------------------------------------------------+
| consulta                                        |
+-------------------------------------------------+
| 3. Distribuição por instituição no trimestre     |
+-------------------------------------------------+
+-----------------------------------+---------+-------------------+------------------+
| instituicao                       | unidade | qtd_distribuicoes | quantidade_total |
+-----------------------------------+---------+-------------------+------------------+
```

Fonte: Captura de tela da execução do script 00_executar_tudo.sql no MariaDB (2026).

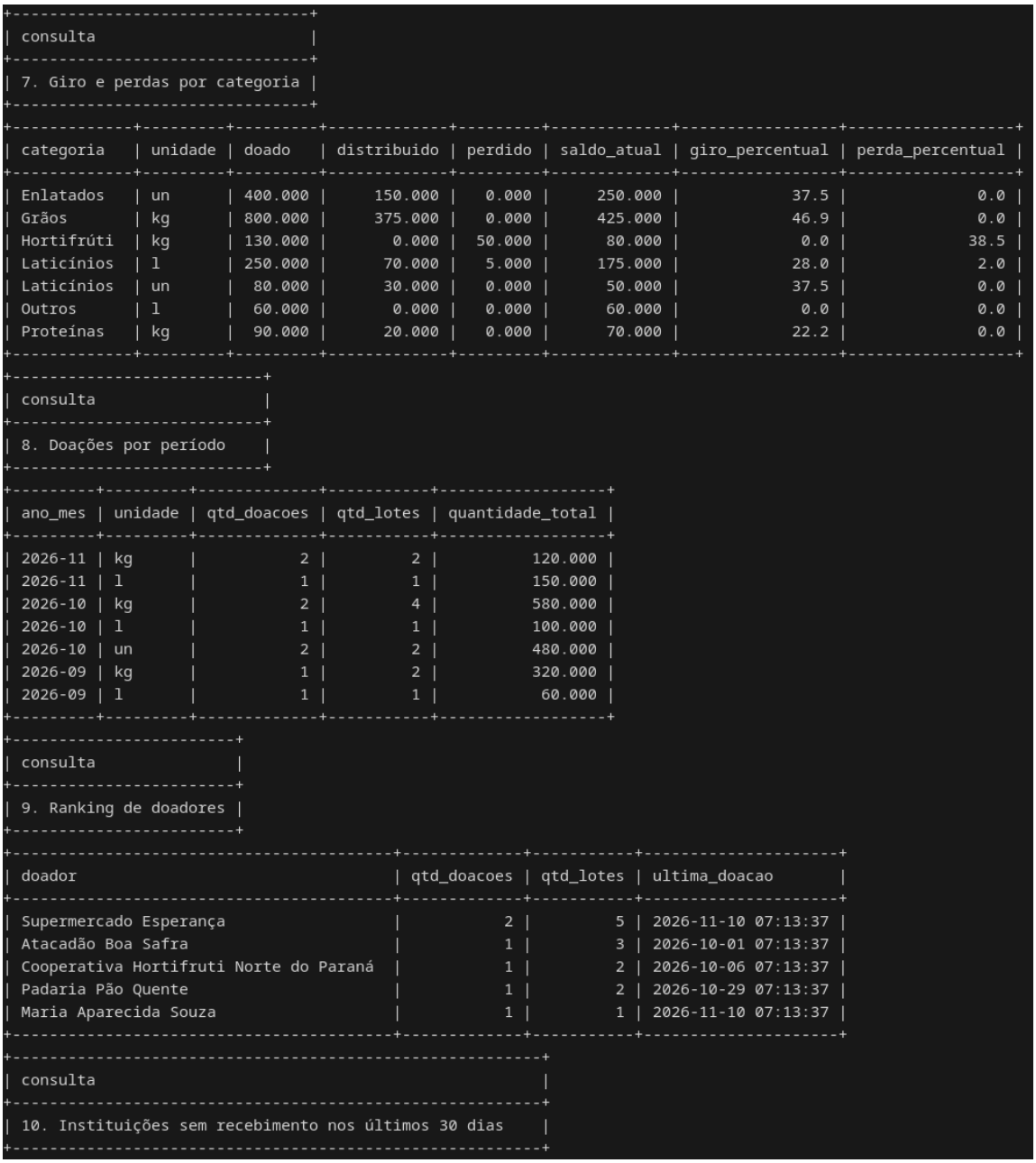
Figura 4 – Execução das consultas 3 a 6



```
+-----------------------------------------------+
| 3. Distribuição por instituição no trimestre  |
+-----------------------------------------------+
+-------------------------------------+---------+-------------------+------------------+
| instituicao                         | unidade | qtd_distribuicoes | quantidade_total |
+-------------------------------------+---------+-------------------+------------------+
| Casa de Acolhimento Esperança Viva  | kg      |                 1 |           40.000 |
| Casa de Acolhimento Esperança Viva  | l       |                 1 |           30.000 |
| Cozinha Solidária Prato Cheio       | kg      |                 2 |          160.000 |
| Cozinha Solidária Prato Cheio       | l       |                 1 |           40.000 |
| Cozinha Solidária Prato Cheio       | un      |                 2 |          180.000 |
| Creche Pequeno Aprendiz             | kg      |                 1 |          105.000 |
+-------------------------------------+---------+-------------------+------------------+
+-----------------------------------------+
| consulta                                |
+-----------------------------------------+
| 4. Lotes que vencem em até 7 dias (alerta)  |
+-----------------------------------------+
+---------+-----------------+---------------------+--------+---------+------------+------------------+
| id_lote | alimento        | doador_origem       | saldo  | unidade | validade   | dias_para_vencer |
+---------+-----------------+---------------------+--------+---------+------------+------------------+
|       9 | Leite Integral  | Padaria Pão Quente  | 25.000 | l       | 2026-11-14 |                4 |
|      10 | Iogurte Natural | Padaria Pão Quente  | 50.000 | un      | 2026-11-16 |                6 |
+---------+-----------------+---------------------+--------+---------+------------+------------------+
+-----------------------------------------------+
| consulta                                      |
+-----------------------------------------------+
| 5. Lotes vencidos com saldo (perda a registrar) |
+-----------------------------------------------+
+---------+--------------+-----------------------------------------+--------+---------+------------+------------------+
| id_lote | alimento     | doador_origem                           | saldo  | unidade | validade   | dias_para_vencer |
+---------+--------------+-----------------------------------------+--------+---------+------------+------------------+
|       7 | Banana Prata | Cooperativa Hortifruti Norte do Paraná  | 50.000 | kg      | 2026-10-11 |              -30 |
|       8 | Tomate       | Cooperativa Hortifruti Norte do Paraná  | 30.000 | kg      | 2026-10-21 |              -20 |
+---------+--------------+-----------------------------------------+--------+---------+------------+------------------+
+------------------------------+
| consulta                     |
+------------------------------+
| 6. Saldo atual por categoria |
+------------------------------+
+-------------+---------+-------------+-----------------+
| categoria   | unidade | saldo_total | lotes_com_saldo |
+-------------+---------+-------------+-----------------+
| Enlatados   | un      |     250.000 |               1 |
| Grãos       | kg      |     425.000 |               5 |
| Laticínios  | l       |     175.000 |               2 |
| Laticínios  | un      |      50.000 |               1 |
| Outros      | l       |      60.000 |               1 |
| Proteínas   | kg      |      70.000 |               1 |
+-------------+---------+-------------+-----------------+
```

Fonte: Captura de tela da execução do script 00_executar_tudo.sql no MariaDB (2026).

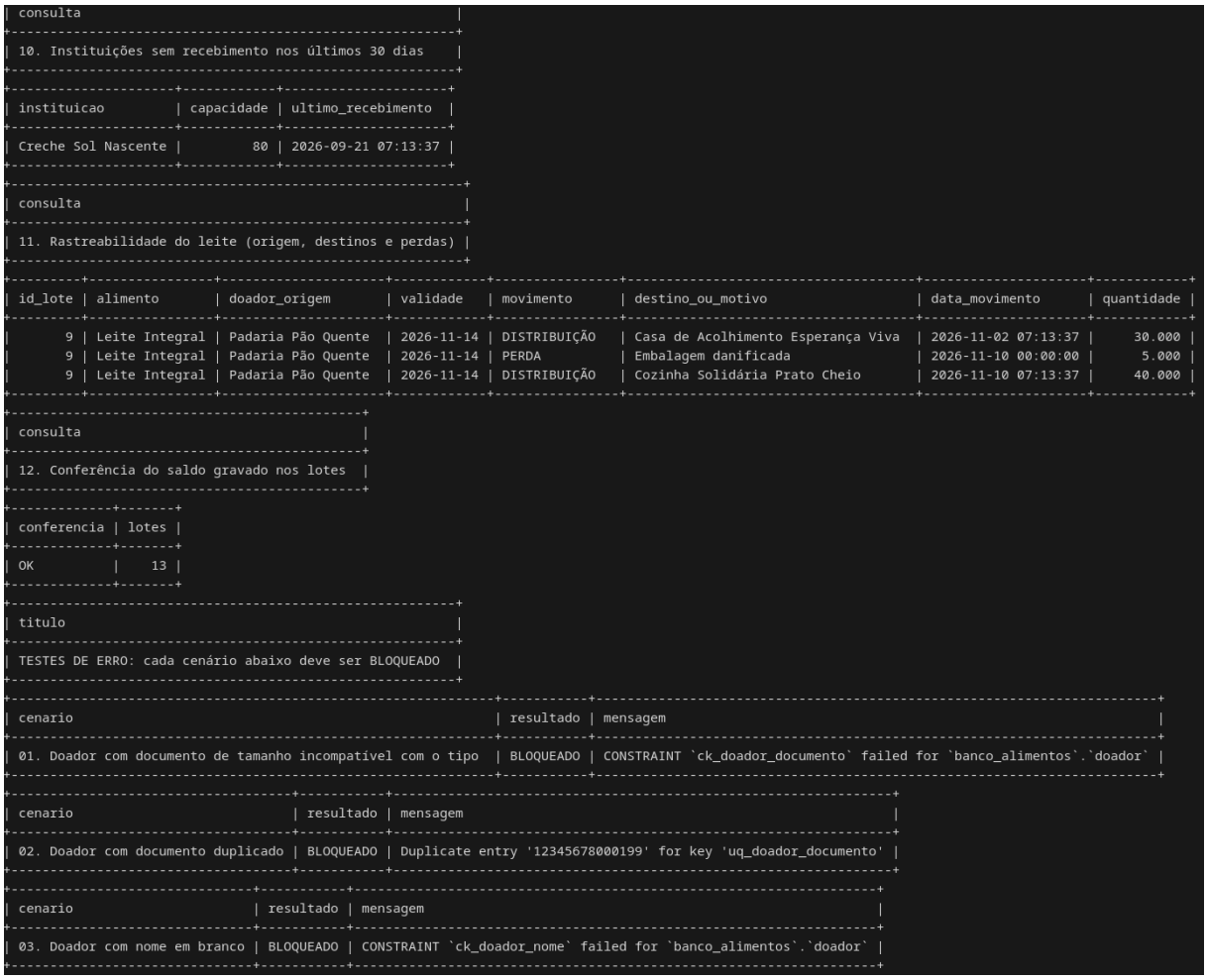
Figura 5 – Execução das consultas 7 a 10



```
+--------------------------------+
| consulta                       |
+--------------------------------+
| 7. Giro e perdas por categoria |
+--------------------------------+
+-------------+---------+---------+-------------+---------+-------------+-----------------+------------------+
| categoria   | unidade | doado   | distribuido | perdido | saldo_atual | giro_percentual | perda_percentual |
+-------------+---------+---------+-------------+---------+-------------+-----------------+------------------+
| Enlatados   | un      | 400.000 |     150.000 |   0.000 |     250.000 |            37.5 |              0.0 |
| Grãos       | kg      | 800.000 |     375.000 |   0.000 |     425.000 |            46.9 |              0.0 |
| Hortifrúti  | kg      | 130.000 |       0.000 |  50.000 |      80.000 |             0.0 |             38.5 |
| Laticínios  | l       | 250.000 |      70.000 |   5.000 |     175.000 |            28.0 |              2.0 |
| Laticínios  | un      |  80.000 |      30.000 |   0.000 |      50.000 |            37.5 |              0.0 |
| Outros      | l       |  60.000 |       0.000 |   0.000 |      60.000 |             0.0 |              0.0 |
| Proteínas   | kg      |  90.000 |      20.000 |   0.000 |      70.000 |            22.2 |              0.0 |
+-------------+---------+---------+-------------+---------+-------------+-----------------+------------------+
+---------------------------+
| consulta                  |
+---------------------------+
| 8. Doações por período    |
+---------------------------+
+---------+---------+-------------+-----------+------------------+
| ano_mes | unidade | qtd_doacoes | qtd_lotes | quantidade_total |
+---------+---------+-------------+-----------+------------------+
| 2026-11 | kg      |           2 |         2 |          120.000 |
| 2026-11 | l       |           1 |         1 |          150.000 |
| 2026-10 | kg      |           2 |         4 |          580.000 |
| 2026-10 | l       |           1 |         1 |          100.000 |
| 2026-10 | un      |           2 |         2 |          480.000 |
| 2026-09 | kg      |           1 |         2 |          320.000 |
| 2026-09 | l       |           1 |         1 |           60.000 |
+---------+---------+-------------+-----------+------------------+
+------------------------+
| consulta               |
+------------------------+
| 9. Ranking de doadores |
+------------------------+
+---------------------------------------+-------------+-----------+---------------------+
| doador                                | qtd_doacoes | qtd_lotes | ultima_doacao       |
+---------------------------------------+-------------+-----------+---------------------+
| Supermercado Esperança                |           2 |         5 | 2026-11-10 07:13:37 |
| Atacadão Boa Safra                    |           1 |         3 | 2026-10-01 07:13:37 |
| Cooperativa Hortifruti Norte do Paraná|           1 |         2 | 2026-10-06 07:13:37 |
| Padaria Pão Quente                    |           1 |         2 | 2026-10-29 07:13:37 |
| Maria Aparecida Souza                 |           1 |         1 | 2026-11-10 07:13:37 |
+---------------------------------------+-------------+-----------+---------------------+
+-------------------------------------------------------+
| consulta                                              |
+-------------------------------------------------------+
| 10. Instituições sem recebimento nos últimos 30 dias  |
+-------------------------------------------------------+
```

Fonte: Captura de tela da execução do script 00_executar_tudo.sql no MariaDB (2026).

Figura 6 – Execução das consultas 10 a 12 e início dos testes de erro



```
| consulta                                                  |
+-----------------------------------------------------------+
| 10. Instituições sem recebimento nos últimos 30 dias      |
+-----------------------------------------------------------+
+----------------------+------------+---------------------+
| instituicao          | capacidade | ultimo_recebimento  |
+----------------------+------------+---------------------+
| Creche Sol Nascente  |         80 | 2026-09-21 07:13:37 |
+----------------------+------------+---------------------+
+-----------------------------------------------------------+
| consulta                                                  |
+-----------------------------------------------------------+
| 11. Rastreabilidade do leite (origem, destinos e perdas)  |
+-----------------------------------------------------------+
+---------+----------------+---------------------+------------+----------------+-----------------------------------+---------------------+------------+
| id_lote | alimento       | doador_origem       | validade   | movimento      | destino_ou_motivo                 | data_movimento      | quantidade |
+---------+----------------+---------------------+------------+----------------+-----------------------------------+---------------------+------------+
|       9 | Leite Integral | Padaria Pão Quente  | 2026-11-14 | DISTRIBUIÇÃO   | Casa de Acolhimento Esperança Viva | 2026-11-02 07:13:37 |     30.000 |
|       9 | Leite Integral | Padaria Pão Quente  | 2026-11-14 | PERDA          | Embalagem danificada              | 2026-11-10 00:00:00 |      5.000 |
|       9 | Leite Integral | Padaria Pão Quente  | 2026-11-14 | DISTRIBUIÇÃO   | Cozinha Solidária Prato Cheio     | 2026-11-10 07:13:37 |     40.000 |
+---------+----------------+---------------------+------------+----------------+-----------------------------------+---------------------+------------+
+-------------------------------------------+
| consulta                                  |
+-------------------------------------------+
| 12. Conferência do saldo gravado nos lotes |
+-------------------------------------------+
+-------------+-------+
| conferencia | lotes |
+-------------+-------+
| OK          |    13 |
+-------------+-------+
+-----------------------------------------------------------+
| titulo                                                    |
+-----------------------------------------------------------+
| TESTES DE ERRO: cada cenário abaixo deve ser BLOQUEADO    |
+-----------------------------------------------------------+
+-----------------------------------------------------------+-----------+-----------------------------------------------------------------------------------+
| cenario                                                   | resultado | mensagem                                                                          |
+-----------------------------------------------------------+-----------+-----------------------------------------------------------------------------------+
| 01. Doador com documento de tamanho incompatível com o tipo | BLOQUEADO | CONSTRAINT `ck_doador_documento` failed for `banco_alimentos`.`doador`            |
+-----------------------------------------------------------+-----------+-----------------------------------------------------------------------------------+
+-----------------------------------+-----------+-----------------------------------------------------------------------+
| cenario                           | resultado | mensagem                                                              |
+-----------------------------------+-----------+-----------------------------------------------------------------------+
| 02. Doador com documento duplicado | BLOQUEADO | Duplicate entry '12345678000199' for key 'uq_doador_documento'        |
+-----------------------------------+-----------+-----------------------------------------------------------------------+
+--------------------------------+-----------+-----------------------------------------------------------------------+
| cenario                        | resultado | mensagem                                                              |
+--------------------------------+-----------+-----------------------------------------------------------------------+
| 03. Doador com nome em branco | BLOQUEADO | CONSTRAINT `ck_doador_nome` failed for `banco_alimentos`.`doador`      |
+--------------------------------+-----------+-----------------------------------------------------------------------+
```

Fonte: Captura de tela da execução do script 00_executar_tudo.sql no MariaDB (2026).

Alguns resultados confirmam as regras do modelo. Na consulta 5, os lotes de banana (50 kg) e tomate (30 kg) da cooperativa aparecem vencidos e com saldo, e a consulta 7 mostra o efeito: Hortifrúti tem 38,5% de perda e giro zero. Na consulta 11, o lote 9 de leite mostra a origem (Padaria Pão Quente), duas distribuições (30 l e 40 l) e uma perda de 5 l por embalagem danificada, o que deixa o saldo em 25 l. A consulta 12 mostra os 13 lotes com saldo conferido (OK).

2.1.10 Testes de integridade

O arquivo 07_testes_erros.sql tenta executar 21 operações inválidas. Cada uma deve ser bloqueada, e a mensagem do banco aparece no resultado. O Quadro 11 resume os cenários e as mensagens obtidas. Nenhum cenário terminou com “FALHOU”. As Figuras 7 a 9 mostram a saída no terminal.

Quadro 11 – Cenários de erro e resultado

Nº	Cenário	Mensagem do banco	Mecanismo
01	Doador com documento de tamanho incompatível com o tipo	Violação de ck_doador_documento	CHECK
02	Doador com documento duplicado	Duplicate entry para uq_doador_documento	UNIQUE
03	Doador com nome em branco	Violação de ck_doador_nome	CHECK
04	Instituição com capacidade zero	Violação de ck_instituicao_capacidade	CHECK
05	Alimento com nome repetido (maiúsculas diferentes)	Duplicate entry “ARROZ TIPO 1” para uq_alimento_nome	UNIQUE
06	Alimento com categoria inexistente	Violação de fk_alimento_categoria	FK
07	Doação sem itens	A doação precisa ter pelo menos um item.	Procedure
08	Doação com lote vencido	Lote vencido: a validade é anterior à data da doação.	Trigger
09	Distribuição com estoque insuficiente	Estoque insuficiente no lote 1: saldo 100.000, pedido 99999.000	Trigger
10	Distribuição de lote vencido	Lote vencido: a validade é anterior à data da saída.	Trigger
11	Distribuição sem itens	A distribuição precisa ter pelo menos um item.	Procedure
12	Distribuição para instituição inexistente	Violação de fk_distribuicao_instituicao	FK
13	Perda maior que o saldo	Estoque insuficiente no lote 1: saldo 100.000, perda 99999.000	Trigger
14	Perda sem motivo	Violação de ck_perda_motivo	CHECK
15	Alterar a validade de um lote	Os dados de origem do lote não podem ser alterados.	Trigger
16	Excluir um lote	Lotes não podem ser excluídos: a origem dos alimentos deve ser preservada.	Trigger
17	Alterar um item de distribuição	Itens de distribuição não podem ser alterados.	Trigger
18	Excluir uma perda	Perdas registradas não podem ser excluídas.	Trigger
19	Excluir doador que já fez doação	Violação de fk_doacao_doador (linha pai referenciada)	FK
20	Atomicidade: segundo item inválido desfaz o primeiro	Estoque insuficiente no lote 1: saldo 99.000, pedido 99999.000	Procedure + trigger
21	Alterar a unidade de um alimento que já tem lote	A unidade de medida não pode ser alterada: o alimento já tem lotes.	Trigger

Fonte: Elaborado para este trabalho (2026).

Ao final, o script confere que os testes não deixaram rastro: o saldo do lote de arroz continua em 100,000 (antes e depois), a quantidade de doações continua 6, a de distribuições continua 5 e os 13 lotes seguem com o saldo conferido (OK). O cenário 20 é o mais importante: o primeiro item da distribuição chegou a descontar 1 unidade

(saldo 99,000 na mensagem), mas o ROLLBACK da procedure desfez a operação inteira.

Figura 7 – Execução dos testes de erro 03 a 11

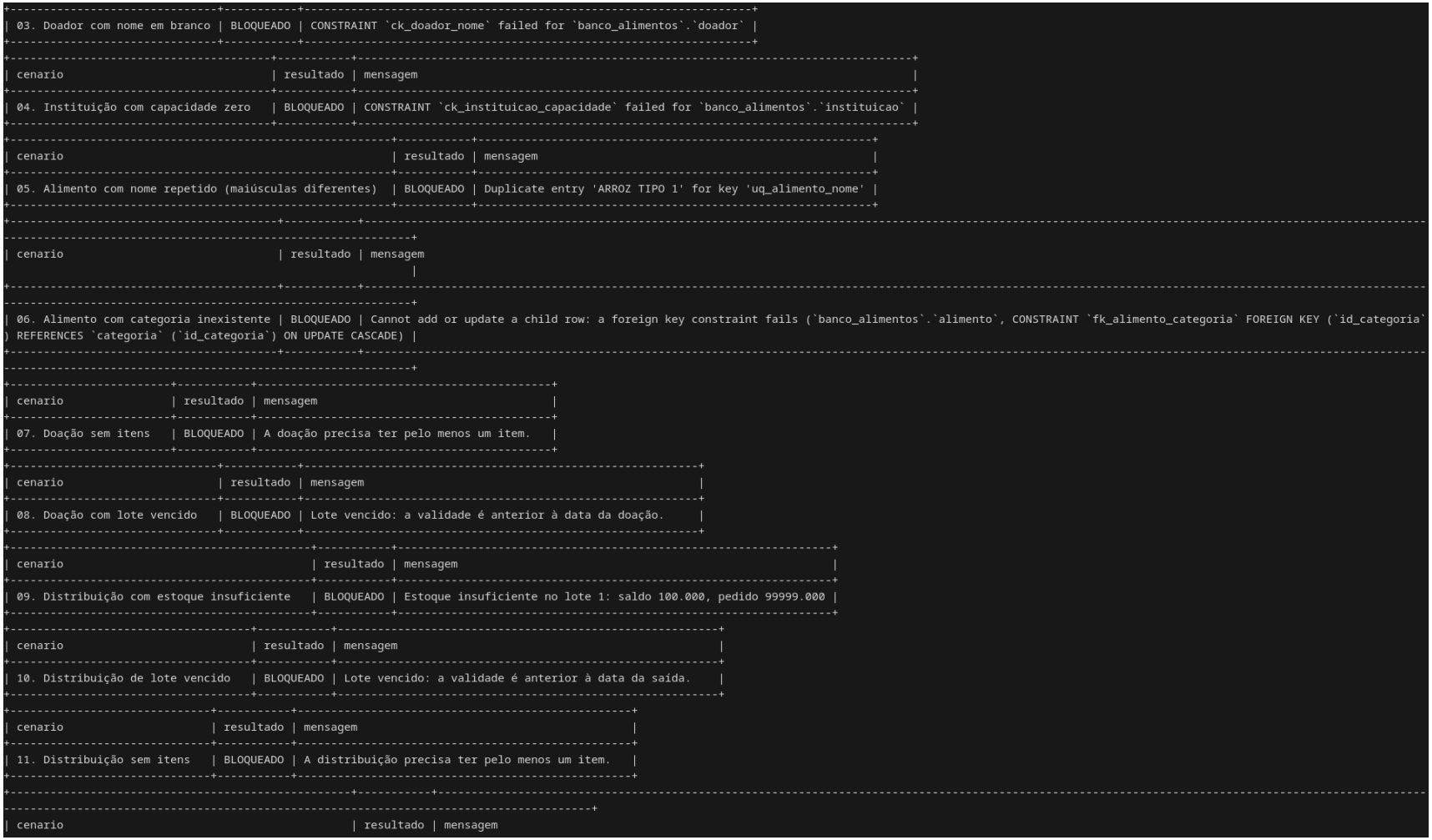
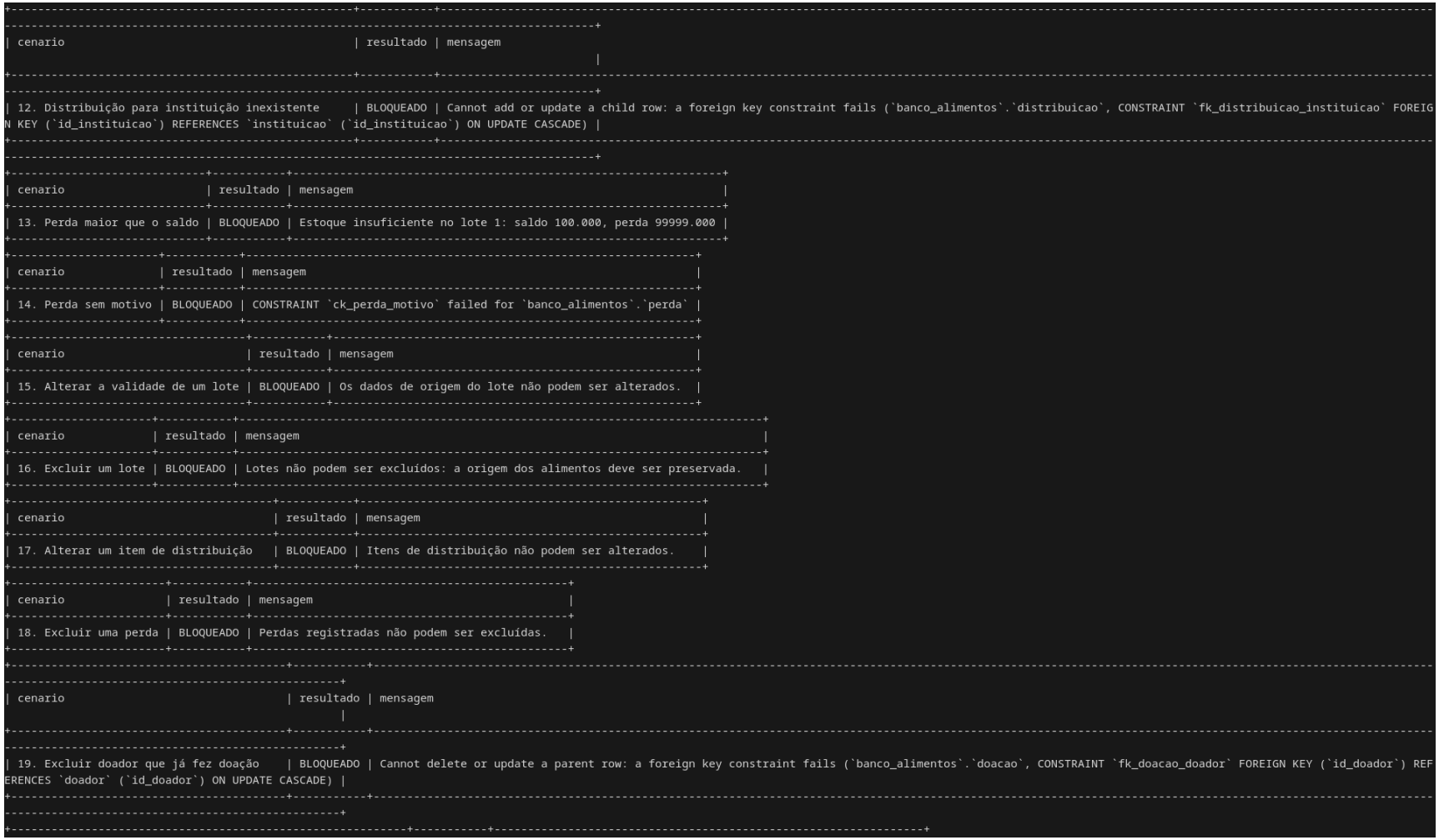
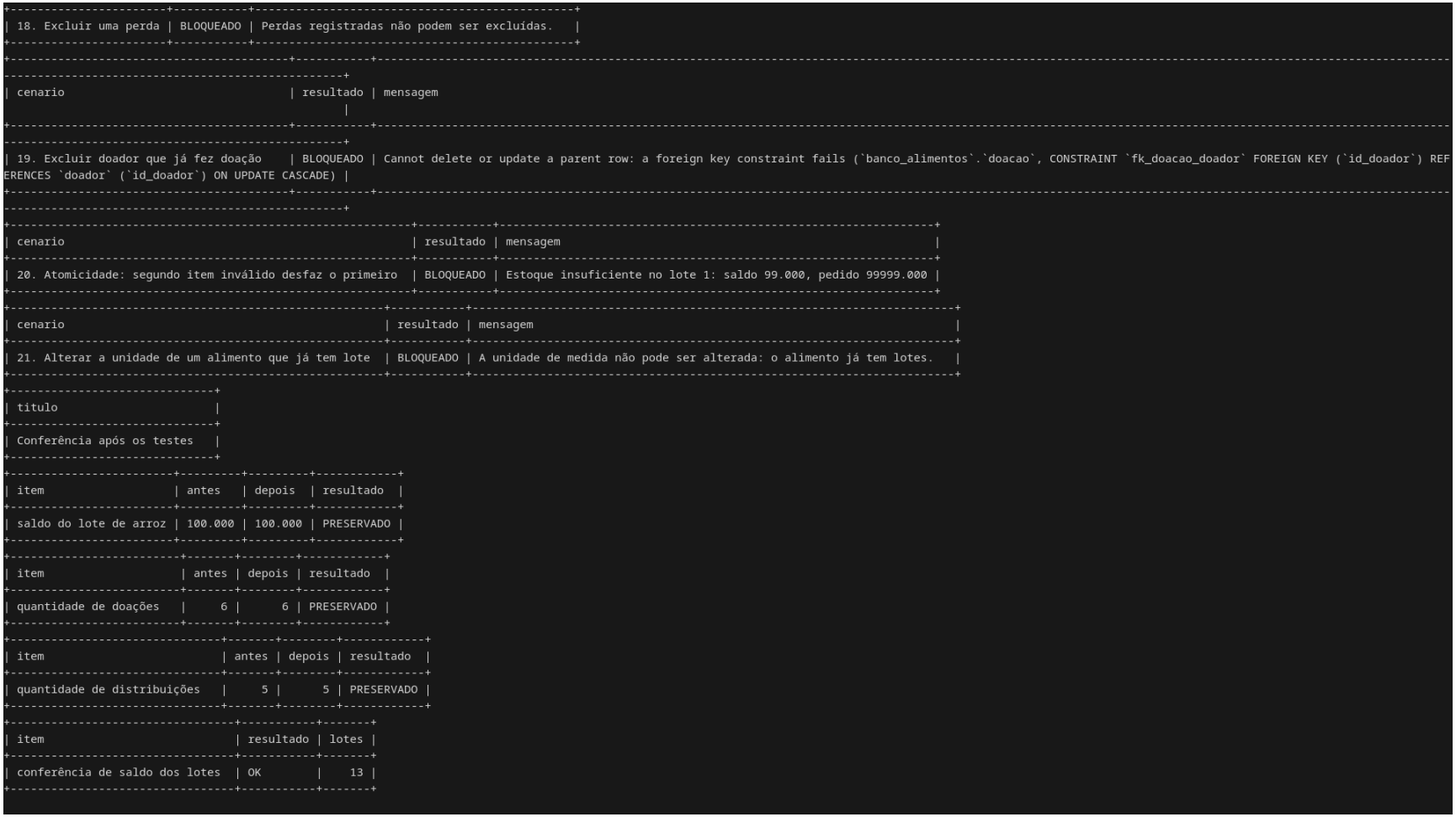

+--------------------------------+-----------+---+
| 03. Doador com nome em branco | BLOQUEADO | CONSTRAINT `ck_doador_nome` failed for `banco_alimentos`.`doador` |
+--------------------------------+-----------+---+
+---------------------------------------+----------+---+
| cenario | resultado | mensagem |
+---------------------------------------+----------+---+
| 04. Instituição com capacidade zero | BLOQUEADO | CONSTRAINT `ck_instituicao_capacidade` failed for `banco_alimentos`.`instituicao` |
+---------------------------------------+-----------+---+
+---+-----------+---+
| cenario | resultado | mensagem |
+---+-----------+---+
| 05. Alimento com nome repetido (maiúsculas diferentes) | BLOQUEADO | Duplicate entry 'ARROZ TIPO 1' for key 'uq_alimento_nome' |
+---+-----------+---+
+---------------------------------------+-----------+---

Figura 8 – Execução dos testes de erro 12 a 19



Fonte: Captura de tela da execução do script 00_executar_tudo.sql no MariaDB (2026).

Figura 9 – Execução dos testes de erro 18 a 21 e conferência final


+----------------------+-----------+--+
| 18. Excluir uma perda | BLOQUEADO | Perdas registradas não podem ser excluídas. |
+----------------------+-----------+--+
+---+-----------+---

2.1.11 Segurança e backup (esboço)

Este item é opcional na 1ª entrega e é apenas uma proposta: os scripts atuais não criam usuários nem rotinas de backup. Para o uso real, sugere-se criar três perfis no SGBD: recepção (inserir doações e consultar estoque), expedição (inserir distribuições e perdas e consultar estoque) e gestão (somente leitura das views e dos indicadores). Nenhum perfil deveria ter permissão de DELETE ou UPDATE direto nas tabelas de histórico, o que reforça os triggers. Para backup, sugere-se uma cópia lógica diária (mysqldump) com RPO desejado de 24 horas e RTO desejado de 4 horas, testando a restauração periodicamente.

2.1.12 Limitações

- O banco ainda não está ligado ao sistema Java. A aplicação usa repositório em memória, mas o acesso passa pela interface CrudRepository, então uma implementação com JDBC poderia ser acrescentada sem mudar as telas e as regras.
- O modelo cobre o núcleo do MVP (doação, estoque, distribuição e perda). Coleta, entrega com rota, usuário e perfil, citados como exemplos no enunciado (UNICESUMAR, 2026, p. 4), não foram modelados nesta versão.
- A regra “pelo menos um item” fica nas procedures. Um INSERT direto em doacao ou distribuicao, sem passar por elas, não é recusado pelo banco.

2.2 SISTEMAS OPERACIONAIS

2.2.1 Objetivo

A partir de uma aplicação Java, o simulador cria tarefas simultâneas em duas situações. A primeira simula um sistema operacional com foco no gerenciamento de threads e na sincronização, representando um galpão logístico com operações de recepção, triagem e expedição protegidas por mutex e semáforos. A segunda aborda os algoritmos de substituição de memória First-In, First-Out (FIFO) e Least Recently Used (LRU), comparando-os nas consultas ao cache de estoque (UNICESUMAR, 2026, p. 6).

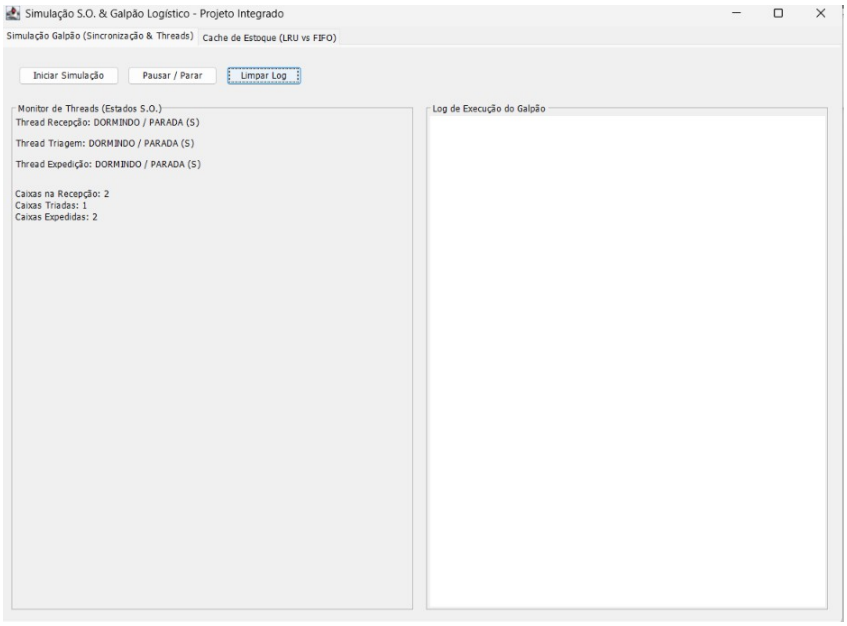
2.2.2 Metodologia

Foi criado um algoritmo na linguagem Java que simula processos de um sistema operacional, o Linux. O fluxo é: a recepção descarrega as caixas, a triagem processa as caixas e a expedição carrega o caminhão.

Os dois algoritmos de cache são submetidos à mesma sequência de acessos: executa-se um cenário com FIFO e, em seguida, repete-se o mesmo cenário com LRU, comparando os resultados. Após a aplicação dos cenários, os dados são analisados considerando hits, misses, total de acessos, hit rate e capacidade do cache.

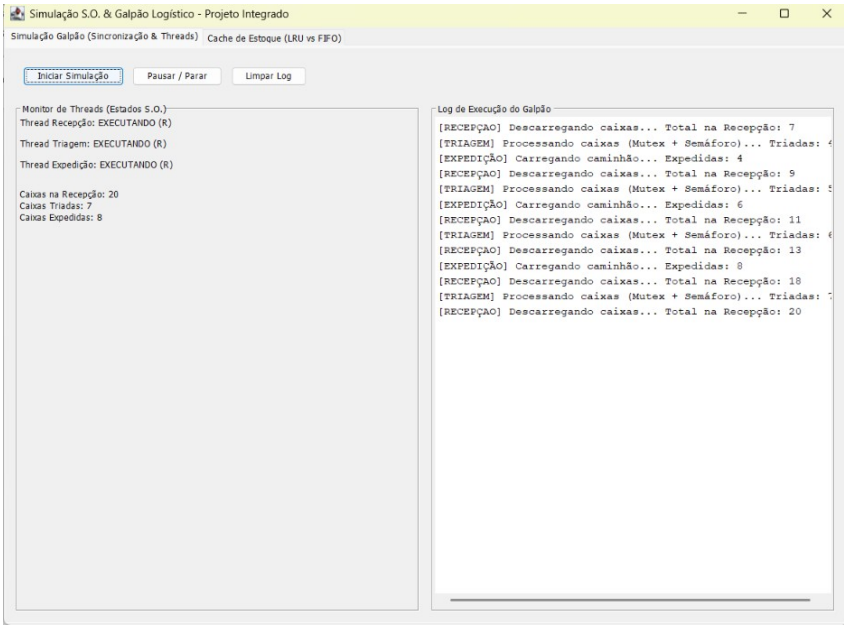
Os dados são observados por meio de uma interface gráfica Swing, nativa do Java, com um monitor de threads inspirado no htop, o visualizador e gerenciador de tarefas do Linux. O monitor mostra o estado de cada thread (executando ou parada) e os contadores de caixas na recepção, triadas e expedidas (Figuras 10 e 11).

Figura 10 – Monitor de threads da simulação, com as threads paradas



Fonte: Captura de tela da simulação do grupo (2026).

Figura 11 – Monitor de threads com a recepção, a triagem e a expedição executando simultaneamente



Fonte: Captura de tela da simulação do grupo (2026).

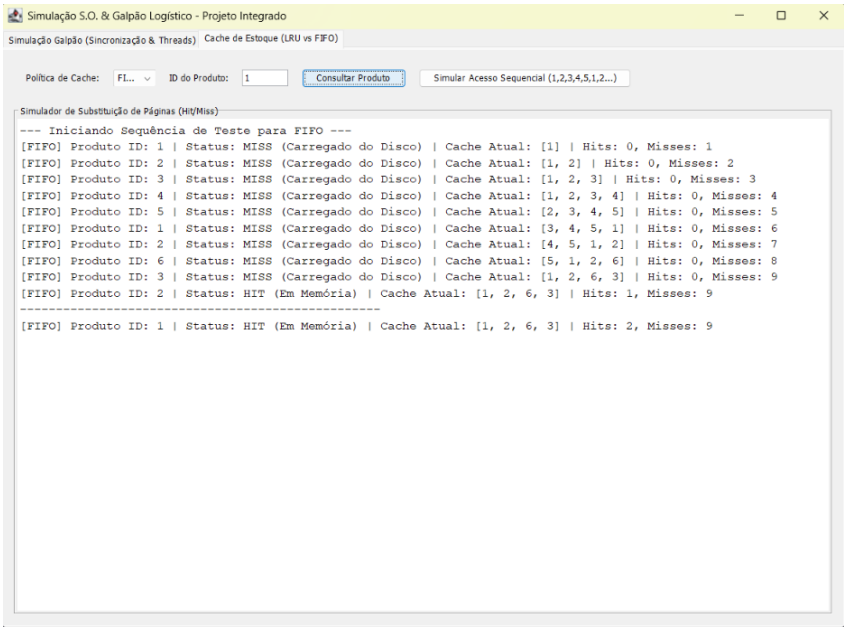
2.2.3 Algoritmos de substituição de páginas

FIFO

O algoritmo FIFO opera seguindo a lógica de uma fila, onde o primeiro item a entrar é obrigatoriamente o primeiro a ser removido ao atingir a capacidade máxima, independentemente de sua frequência de uso. Em Sistemas Operacionais e gestão de

estoque, essa estratégia prioriza a ordem cronológica de chegada dos dados ou itens (GEEKSFORGEEKS, 2026a; IBM, 2026b).

Figura 12 – Rastreando e simulando o produto com ID 1 com o FIFO

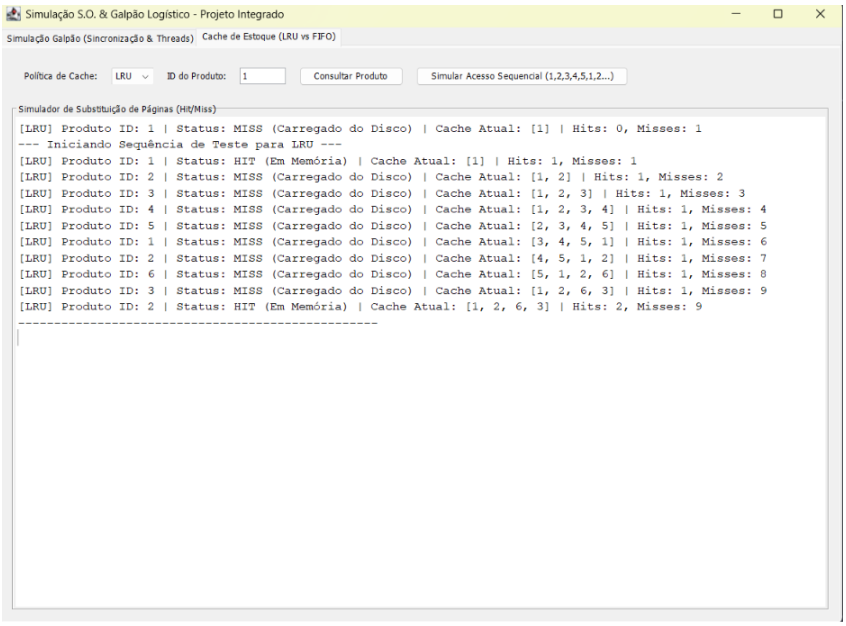


Fonte: Captura de tela da simulação do grupo (2026).

LRU

O algoritmo LRU prioriza a substituição de dados que não são acessados há mais tempo, garantindo que os itens mais requisitados permaneçam na memória cache. Esta estratégia otimiza a taxa de acertos (hit rate) ao explorar a localidade de referência, sendo superior ao modelo FIFO em diversos cenários práticos. O funcionamento do LRU baseia-se na manutenção de uma lista onde o dado mais recentemente acessado é movido para o topo, enquanto o elemento no final da fila é o escolhido para remoção quando o cache atinge sua capacidade máxima. Essa técnica é fundamental para o gerenciamento eficiente de memória e cache em sistemas computacionais (GEEKSFORGEEKS, 2026a; IBM, 2026b).

Figura 13 – Rastreando e simulando o produto com ID 1 com o LRU



Fonte: Captura de tela da simulação do grupo (2026).

FIFO versus LRU

Embora o FIFO seja mais simples de ser implementado, o desempenho pode ser afetado, pois só leva em consideração a ordem de chegada como fator, sendo assim, pode ocorrer a condição em que um processo utilizado várias vezes seja excluído apenas por ter sido o primeiro a entrar na memória cache, independentemente do número de vezes de sua utilização.

O algoritmo LRU pode ser mais eficiente nesse cenário, pois considera o tempo decorrido desde o último acesso a cada elemento, removendo aquele que está há mais tempo sem ser utilizado. Dessa maneira, os itens acessados mais recentemente têm maior probabilidade de permanecer na memória cache, que é uma memória de acesso rápido utilizada para armazenar dados. Além disso, a posição dos elementos na fila é atualizada conforme novos acessos acontecem, mantendo os dados mais recentemente utilizados no início e os menos recentemente acessados no final.

2.2.4 Mutex e semáforos

Mutex

O Mutex, Mutual Exclusion, é um mecanismo de sincronização utilizado em sistemas operacionais. O seu objetivo é a exclusão mútua no acesso a recursos compartilhados, ou seja, que apenas uma thread seja executada por vez em uma seção crítica e, por consequência, que se evite a condição de corrida. Quando uma thread adquire um Mutex, as demais threads que tentarem acessar o mesmo recurso

deverão aguardar até que ele seja liberado. Após a liberação, outra thread poderá adquirir o bloqueio e executar sua operação (JIANG, 2017).

Semáforos

Os semáforos são mecanismos de sincronização utilizados em sistemas operacionais para controlar o acesso simultâneo a recursos compartilhados. Seu principal objetivo é limitar a quantidade de threads que podem acessar determinado recurso ao mesmo tempo, evitando conflitos durante a execução das tarefas (ORACLE, 2026c).

Diferentemente do Mutex, que permite apenas uma thread por vez em uma seção crítica, os semáforos podem permitir que várias threads acessem um recurso simultaneamente, respeitando um limite definido. Quando esse limite é atingido, as demais threads precisam aguardar até que uma das posições seja liberada (MICROSOFT, 2025).

2.2.5 Prevenção de condições de corrida e deadlock

No simulador, as três threads (Thread-Recepcao, Thread-Triagem e Thread-Expedicao) compartilham os contadores de caixas na recepção, triadas e expedidas. Sem proteção, duas threads poderiam ler e alterar o mesmo contador ao mesmo tempo, e uma das atualizações seria perdida. Essa situação é uma condição de corrida (GEEKSFORGEEKS, 2026b).

Para evitar isso, todo acesso aos contadores acontece dentro de uma seção crítica protegida por um mutex (ReentrantLock). A thread adquire o mutex com lock() e o libera em um bloco finally, de modo que ele é devolvido mesmo quando a thread é interrompida. Nenhuma thread dorme com o mutex adquirido: o tempo de espera de cada etapa (2, 3 e 4 segundos) ocorre depois da liberação.

Os semáforos (Semaphore) definem quantas threads de cada etapa podem entrar na seção crítica ao mesmo tempo: duas permissões na triagem e uma na expedição. Nesta simulação há uma thread por etapa, então o semáforo funciona como autorização de entrada, e o limite já está definido para o caso de a etapa ganhar mais threads.

O deadlock não ocorre porque existe um único mutex, o que elimina a espera circular entre travas, e porque todas as threads obedecem à mesma ordem de aquisição: primeiro o semáforo e depois o mutex. Além disso, os dois são liberados em blocos finally, e só se liberam os recursos que foram de fato adquiridos.

A proteção foi verificada observando a consistência dos contadores durante a execução. A cada ciclo, a recepção acrescenta 5 caixas, a triagem move 3 da

recepção para as triadas e a expedição move 2 das triadas para as expedidas, então a soma dos três contadores é sempre igual ao total descarregado, que cresce de 5 em 5. Na execução mostrada na Figura 11, os valores 20, 7 e 8 somam 35, que é múltiplo de 5.

2.2.6 Análise dos dados

Quadro 12 – Resultados da simulação com FIFO e LRU

Indicador	FIFO	LRU
Hits	2	2
Misses	9	9
Total de acessos	11	11
Hit rate	18,18%	18,18%
Capacidade do cache	4	4

Fonte: Dados gerados pela simulação do grupo (2026), com a sequência pré-definida do simulador (1, 2, 3, 4, 5, 1, 2, 6, 3, 2) e uma consulta manual ao produto de ID 1.

Após a simulação, notamos um empate em todos os quesitos avaliados, com hit rate idêntico de 18,18% (2 acertos em 11 acessos). Nessa sequência, os dois algoritmos mantiveram os mesmos itens no cache em todos os acessos, porque os itens repetidos só voltam a ser consultados depois de já terem sido substituídos nos dois casos. Por isso o FIFO, que considera a ordem de entrada dos elementos, e o LRU, que considera o tempo desde o último acesso, produziram o mesmo resultado. Sequências com acessos repetidos em curto intervalo podem favorecer o LRU, mas esta simulação não testou esse padrão.

2.2.7 Conclusão da disciplina

A partir desta atividade, podemos concluir que o sistema operacional desempenha um papel fundamental no controle de recursos, seja garantindo a sincronização e a exclusão mútua de tarefas por meio de Mutex e Semáforos, ou otimizando o uso de recursos com algoritmos de gerenciamento de memória (FIFO e LRU). No entanto, apesar de os resultados serem iguais, não foi possível determinar qual algoritmo possui maior eficiência, pois cada um deles leva em consideração um critério.

2.3 PROGRAMAÇÃO ORIENTADA A OBJETOS

2.3.1 Proposta, usuários e glossário

O banco de alimentos usava planilhas para acompanhar as doações. Isso dificultava saber de onde veio cada alimento, quando vence e quanto ainda pode ser distribuído. A proposta é organizar essas informações em um sistema Java com cadastro de doadores, instituições e alimentos, registro de doações em lotes, distribuição para instituições, registro de perdas e consulta do estoque por validade. O contexto e os itens da atividade seguem o enunciado da Unicesumar (UNICESUMAR, 2026, p. 1-4).

Os objetivos do sistema são: saber a origem, a validade e o destino de cada alimento; impedir saídas inválidas (lote vencido ou sem saldo); e manter separadas a apresentação, as regras de negócio e a persistência.

Os usuários previstos são a recepção, que cadastra doadores e registra entradas; a triagem, que confere os lotes; a expedição, que registra saídas; e o gestor, que acompanha a operação. O Quadro 13 define os termos usados no projeto.

Quadro 13 – Glossário

Termo	Significado
Doador	Pessoa física ou jurídica que fornece alimentos.
Instituição	Creche, casa de acolhimento ou cozinha solidária que recebe alimentos.
Alimento	Tipo de alimento cadastrado, com categoria e unidade de medida.
Lote	Reúne alimento, doador de origem, validade e saldo. É criado por um item de doação.
Saldo	Quantidade ainda disponível em um lote.
Doação	Entrada de alimentos, com um ou mais itens.
Distribuição	Saída de alimentos de um ou mais lotes para uma instituição.
Perda	Baixa de parte de um lote, com quantidade e motivo.
CRUD	Cadastrar, consultar, atualizar e excluir.

Fonte: Elaborado para este trabalho (2026).

2.3.2 Requisitos funcionais e não funcionais

O Quadro 14 lista os requisitos. Os códigos POO-RF01 a POO-RF05 e POO-RNF01 a POO-RNF04 são os da 1ª entrega e foram mantidos; os demais foram acrescentados na 2ª entrega, sem reaproveitar códigos.

Quadro 14 – Requisitos do sistema

Código	Requisito
POO-RF01	Cadastrar, listar, atualizar e excluir doadores.
POO-RF02	Registrar doação com doador, alimento, quantidade e validade, com um ou mais itens.
POO-RF03	Consultar lotes em ordem de validade, mostrando saldo, origem, dias para vencer e situação.
POO-RF04	Distribuir alimentos para uma instituição e reduzir o saldo dos lotes.
POO-RF05	Informar erros e impedir distribuição inválida.
POO-RF06	Cadastrar, listar, atualizar e excluir instituições.
POO-RF07	Cadastrar, listar, atualizar e excluir alimentos, com nome, categoria e unidade de medida.
POO-RF08	Registrar perda de um lote, com quantidade e motivo, reduzindo o saldo.
POO-RNF01	Usar Java e separar interface, regras de negócio e persistência.
POO-RNF02	Exibir campos e mensagens em português.
POO-RNF03	Preservar os dados anteriores quando uma operação falhar.
POO-RNF04	Manter testes automatizados para cadastro e movimentação de estoque.
POO-RNF05	Oferecer interface gráfica integrada às regras de negócio, sem regra de negócio nas telas.

Fonte: Elaborado para este trabalho (2026).

2.3.3 Regras de negócio

Quadro 15 – Regras de negócio

Código	Regra
POO-RN01	Nome, documento, contato e tipo do doador são obrigatórios. O documento deve ser único.
POO-RN02	Todo lote precisa de alimento, doador de origem e validade. Quantidades informadas devem ser positivas.
POO-RN03	Não é permitido retirar mais que o saldo disponível, nem distribuir um lote cuja validade seja anterior ao dia da saída.
POO-RN04	Toda distribuição precisa de uma instituição destinatária. Uma operação concluída precisa ter pelo menos um item.
POO-RN05	A exclusão exige confirmação. Uma atualização recusada deve manter os dados anteriores.
POO-RN06	Doador com lote no estoque, alimento com lote no estoque e instituição com distribuição registrada não podem ser excluídos.
POO-RN07	Uma perda exige quantidade positiva, motivo informado e saldo suficiente no lote.
POO-RN08	O documento de instituição é único, ignorando pontuação, e o nome do alimento é único, ignorando maiúsculas e minúsculas.
POO-RN09	Todo alimento tem uma unidade de medida, escolhida entre kg, g, l, ml e un. A unidade de um alimento que já tem lotes no estoque não pode ser alterada.

Fonte: Elaborado para este trabalho (2026).

2.3.4 Casos de uso e critérios de aceitação

Quadro 16 – Casos de uso e critérios de aceitação

Caso de uso	Ator	Critérios de aceitação
POO-UC01 Manter doador	Recepção	O registro aparece na lista, a edição mantém o identificador e a exclusão, depois de confirmada, remove o cadastro. Dados vazios ou documento repetido bloqueiam a gravação.
POO-UC02 Registrar doação	Recepção	O usuário informa doador, alimento, quantidade e validade, adiciona os itens e registra. Cada item cria um lote com origem e saldo. Doação sem itens, quantidade não positiva ou validade anterior à data da doação bloqueiam a entrada.
POO-UC03 Consultar estoque	Triagem	Os lotes aparecem em ordem de validade, com saldo, origem, dias para vencer e situação. Sem registros, a tabela fica vazia.
POO-UC04 Distribuir alimento	Expedição	O usuário escolhe a instituição, seleciona lotes disponíveis e informa as quantidades. Saldo 10 com retirada 4 resulta em saldo 6. Lote vencido, quantidade inválida ou saldo insuficiente bloqueiam a saída inteira, sem alterar nenhum saldo.
POO-UC05 Manter instituição	Recepção	Como no POO-UC01. O documento é único, ignorando pontuação. Instituição com distribuição registrada não pode ser excluída.
POO-UC06 Manter alimento	Recepção	Como no POO-UC01. A unidade de medida é escolhida em uma lista (kg, g, l, ml ou un). O nome é único, ignorando maiúsculas e minúsculas. Alimento com lote no estoque não pode ser excluído nem mudar de unidade.
POO-UC07 Registrar perda	Triagem	O usuário seleciona um lote no estoque, informa quantidade e motivo, e o saldo diminui. Perda maior que o saldo ou sem motivo é recusada.

Fonte: Elaborado para este trabalho (2026).

2.3.5 Visão geral da 2ª entrega

Nesta entrega o sistema deixou de ser só o modelo de domínio. Agora ele tem CRUD completo, interface gráfica e testes automatizados. O código é em Java e está separado em três partes: apresentação (telas), regras de negócio (entidades e serviços) e persistência (repositório). A persistência é em memória, então os dados são perdidos quando o programa fecha. Os itens abaixo seguem o que o enunciado pede para a 2ª entrega de POO (UNICESUMAR, 2026, p. 6-7).

O Quadro 17 mostra o que ficou pendente na 1ª entrega e como está agora.

Quadro 17 – Pendências da 1ª entrega e situação atual

Pendência da 1ª entrega	Situação atual
CRUD de doadores apenas descrito	CRUD implementado para doadores, instituições e alimentos
Registro de doações sem formulário	Tela de doação com vários itens por doação
Instituição fixa na distribuição	Instituição escolhida na tela de distribuição

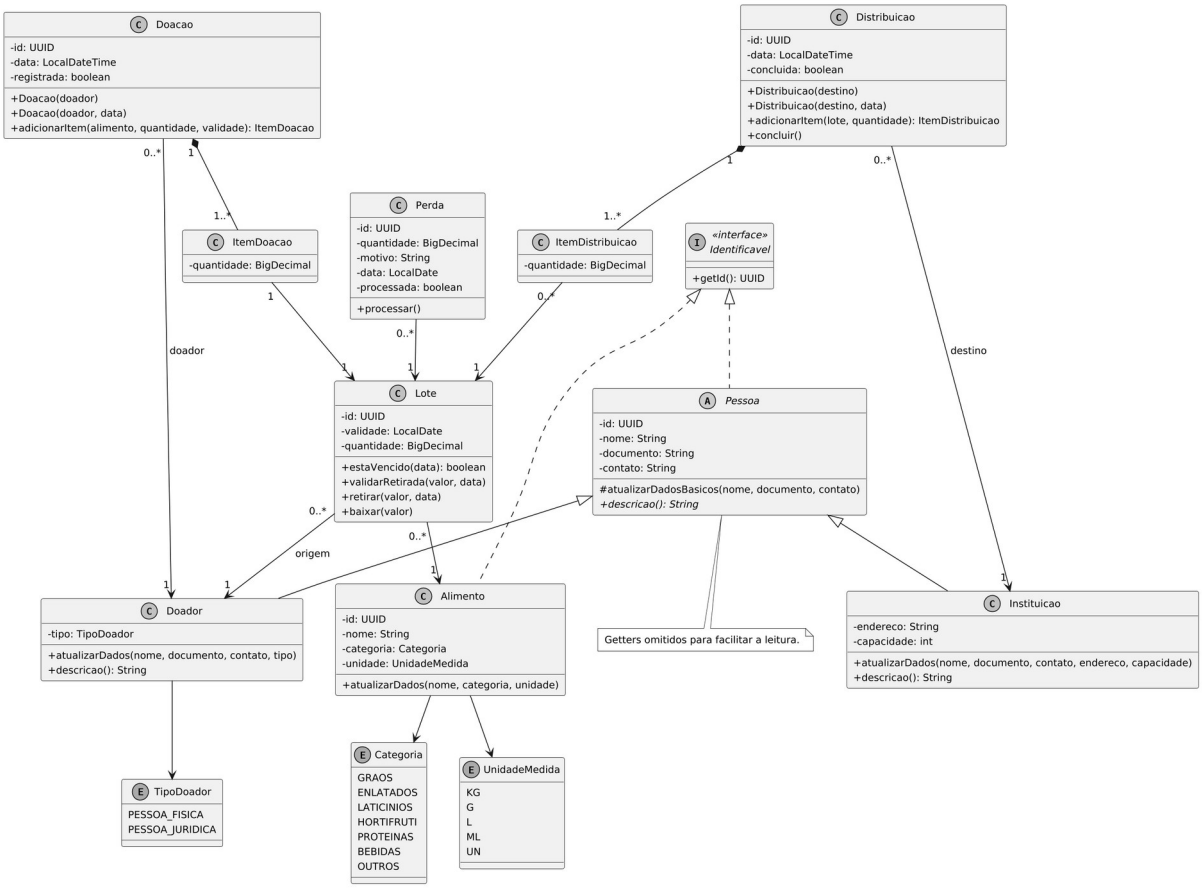
Pendência da 1ª entrega	Situação atual
Alteração parcial do cadastro	Os dados são validados antes de qualquer alteração e há teste para isso
Composição sem proteção no código	Construtores de lote e dos itens sem modificador público; só Doacao e Distribuicao criam itens
Testes não executados	45 testes JUnit executados, todos aprovados

Fonte: Elaborado para este trabalho (2026).

2.3.6 Diagrama de classes refatorado

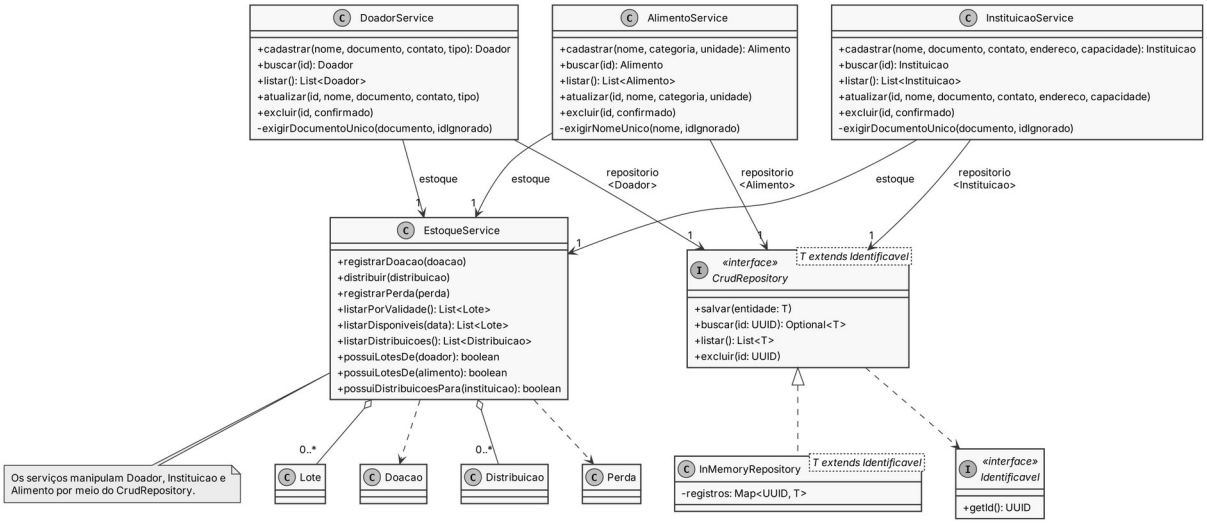
O diagrama foi dividido em quatro figuras para ficar legível. A Figura 14 mostra o domínio, a Figura 15 os serviços e a persistência, a Figura 16 a camada de apresentação e a Figura 17 as exceções.

Figura 14 – Diagrama de classes do domínio



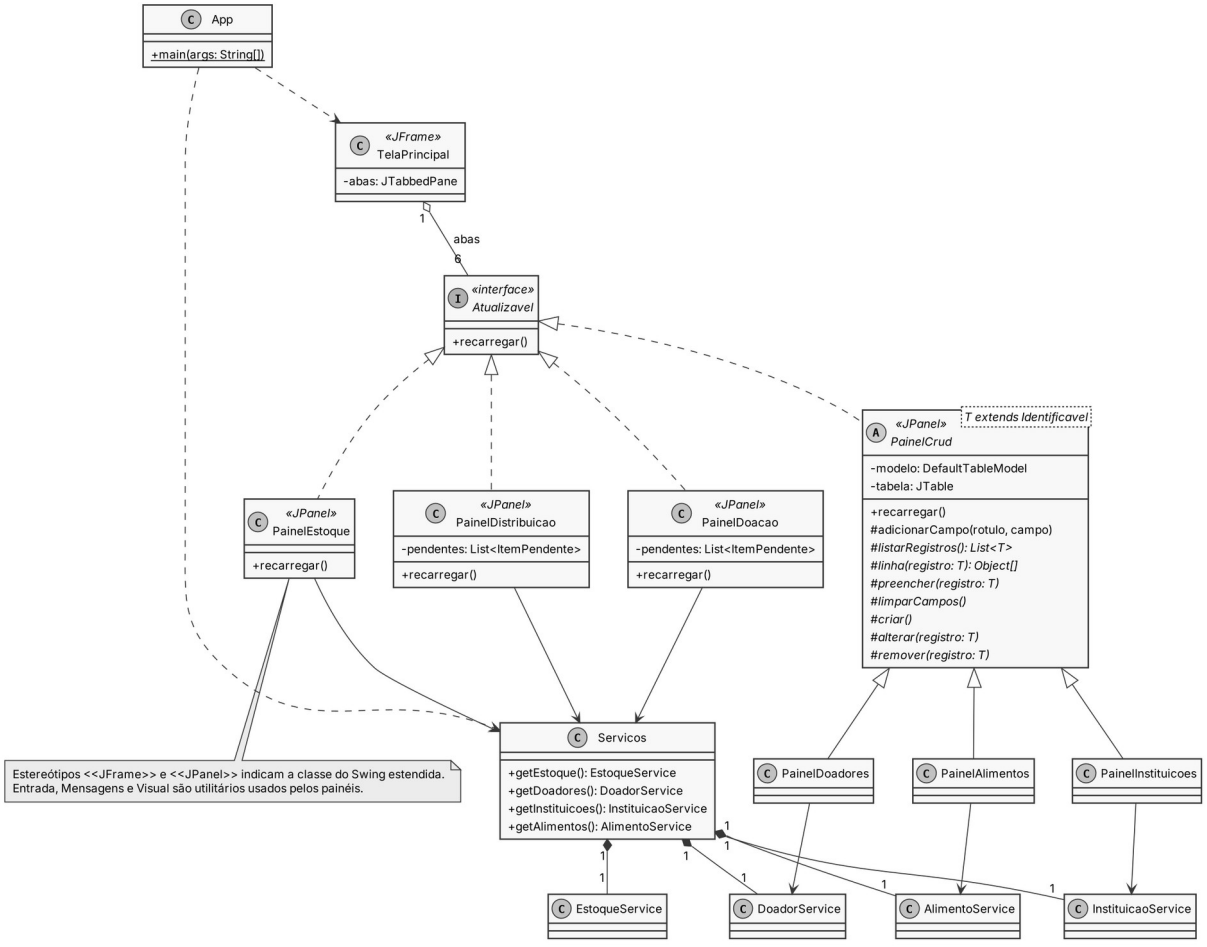
Fonte: Elaborado para este trabalho (2026).

Figura 15 – Diagrama de classes dos serviços e da persistência



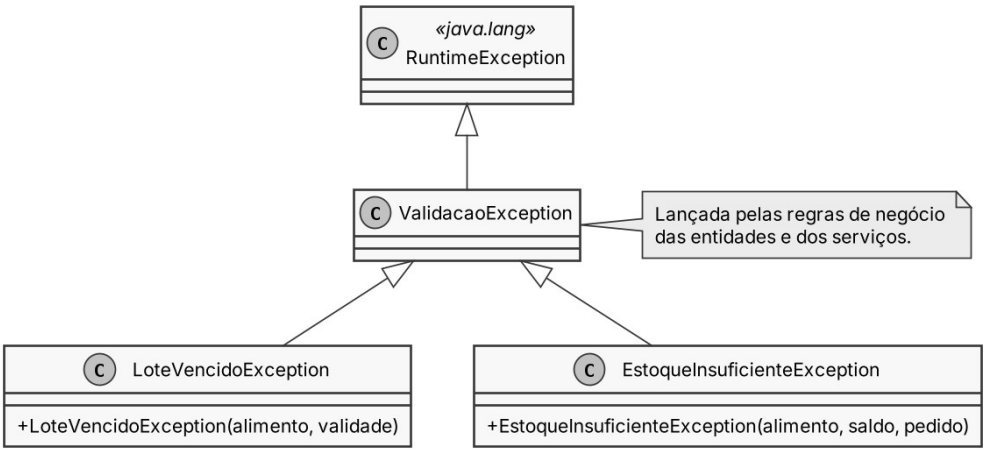
Fonte: Elaborado para este trabalho (2026).

Figura 16 – Diagrama de classes da apresentação



Fonte: Elaborado para este trabalho (2026).

Figura 17 – Hierarquia das exceções



Fonte: Elaborado para este trabalho (2026).

Alterações em relação ao diagrama da 1ª entrega:

- Nova classe de domínio Perda, que registra a baixa de um lote com quantidade e motivo.
- Novas enums Categoria, TipoDoador e UnidadeMedida. A unidade do alimento deixou de ser texto livre.
- Novas interfaces Identificavel e CrudRepository. O InMemoryDoadorRepository foi substituído por um InMemoryRepository genérico, usado por doadores, instituições e alimentos.
- Nova camada de serviços: DoadorService, InstituicaoService e AlimentoService, cada um com cadastrar, buscar, listar, atualizar e excluir.
- Instituicao e Alimento ganharam o método atualizarDados, e Alimento passou a implementar Identificavel.
- O EstoqueService passou a guardar as distribuições e a informar se um doador, um alimento ou uma instituição já tem lotes ou distribuições ligados a ele.
- A multiplicidade dos itens de doação e de distribuição passou de 0..* para 1..*. Uma operação sem itens é recusada ao registrar.
- Nova camada de apresentação em Swing, com a classe abstrata PainelCrud e a interface Atualizavel.

2.3.7 Implementação das classes e associações

A Doacao é composta por itens (ItemDoacao). Cada item guarda o Lote que ele criou, e o lote conhece o Alimento e o Doador de origem. A Distribuicao também é composta por itens (ItemDistribuicao), que apontam para lotes já existentes, e aponta para a Instituicao de destino. A Perda aponta para o lote que sofreu a baixa. O EstoqueService agrega os lotes e as distribuições.

Os atributos são privados. As listas de itens são devolvidas como listas não modificáveis. O saldo de um lote só diminui pelos métodos retirar (saída com validade) e baixar (perda), e os dois conferem a quantidade antes. Os construtores de Lote, ItemDoacao e ItemDistribuicao não são públicos, para que esses objetos só nasçam dentro de Doacao e Distribuicao.

As operações que alteram vários dados validam tudo antes de mudar qualquer coisa. Por exemplo, a Distribuicao confere todos os lotes e só depois baixa os saldos. Se um item falha, nenhum saldo muda. A atualização de Pessoa e Instituicao também valida todos os campos antes de gravar.

2.3.8 Herança, polimorfismo, sobrecarga, interfaces e classes abstratas

Quadro 18 – Conceitos de orientação a objetos aplicados

Conceito	Onde aparece no projeto
Classe abstrata	Pessoa, com o método abstrato descricao(); PainelCrud, com os métodos que cada tela de CRUD precisa preencher
Herança	Doador e Instituicao herdam de Pessoa; PainelDoadores, PainelInstituicoes e PainelAlimentos herdam de PainelCrud
Polimorfismo	Cada subclasse de Pessoa tem a sua descricao(), usada pelo toString(); a TelaPrincipal chama recarregar() em qualquer painel Atualizavel sem saber qual é
Sobrecarga	Doacao(doador) e Doacao(doador, data); Distribuicao(destino) e Distribuicao(destino, data); possuiLotesDe(Doador) e possuiLotesDe(Alimento)
Interfaces	Identificavel (getId), CrudRepository (salvar, buscar, listar, excluir) e Atualizavel (recarregar)
Generics	CrudRepository, InMemoryRepository e PainelCrud, todos limitados a tipos Identificavel
Composição	Doacao com ItemDoacao; Distribuicao com ItemDistribuicao
Agregação	EstoqueService reúne lotes e distribuições
Encapsulamento	Atributos privados, listas não modificáveis e construtores restritos
Enumerações	Categoria, TipoDoador e UnidadeMedida (kg, g, l, ml, un), que impedem valores livres
Template Method	PainelCrud define o fluxo de salvar, novo e excluir; as subclasses implementam criar, alterar e remover

Fonte: Elaborado para este trabalho (2026).

2.3.9 Coleções, exceções e validações

O sistema usa List para itens, lotes e distribuições, e Map para os registros do repositório (por identificador) e para somar as quantidades por lote ao concluir uma distribuição. As buscas devolvem Optional.

Os erros de regra de negócio são exceções. A ValidacaoException é a base, e dela saem a LoteVencidoException (Figura 17) e a EstoqueInsuficienteException. As validações ficam nas entidades e nos serviços, não nas telas. A tela só mostra a mensagem. A classe Entrada converte os textos digitados (números e datas) e lança ValidacaoException quando o valor é inválido. As regras aplicadas pelo código são as POO-RN01 a POO-RN09 do Quadro 15.

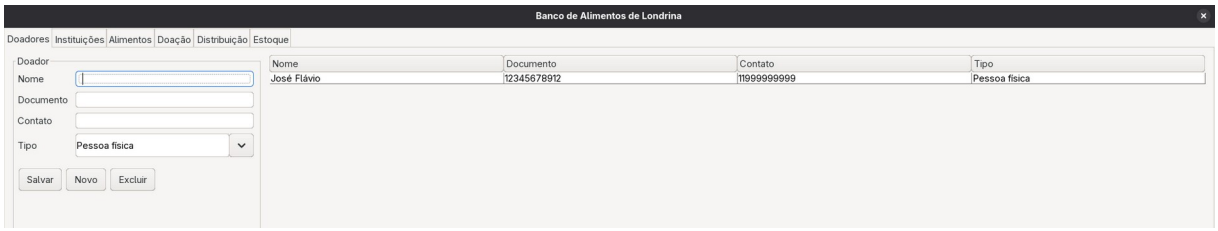
2.3.10 Interface gráfica

A interface é em Swing e roda pela classe App. Ela tem seis abas:

- Doadores, Instituições e Alimentos: tabela com os registros e formulário para cadastrar, editar e excluir. A exclusão pede confirmação.
- Doação: o usuário escolhe doador e alimento, informa quantidade e validade, adiciona os itens e registra. Cada item vira um lote no estoque.
- Distribuição: o usuário escolhe a instituição, seleciona lotes disponíveis (ordenados por validade), informa a quantidade e conclui.
- Estoque: lista os lotes com saldo, validade, dias para vencer e situação (OK, vence em até 7 dias, vencido ou esgotado). Também registra perdas.

As telas não têm regra de negócio. Elas chamam os serviços, que ficam reunidos na classe Servicos. Assim a apresentação, as regras e a persistência continuam separadas. Quando uma regra é violada, a tela mostra a mensagem da exceção. As Figuras 18 a 23 mostram as seis abas, com dados de exemplo cadastrados.

Figura 18 – Aba Doadores



Fonte: Captura de tela do sistema (2026).

Figura 19 – Aba Instituições



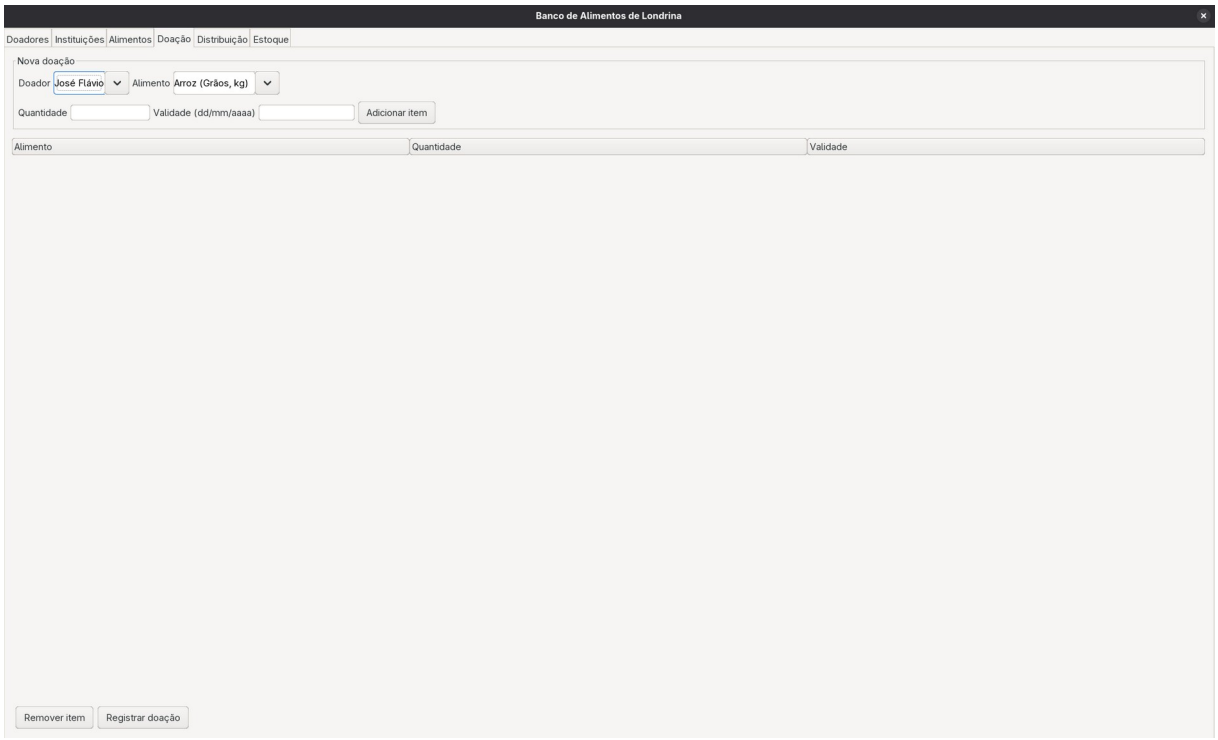
Fonte: Captura de tela do sistema (2026).

Figura 20 – Aba Alimentos



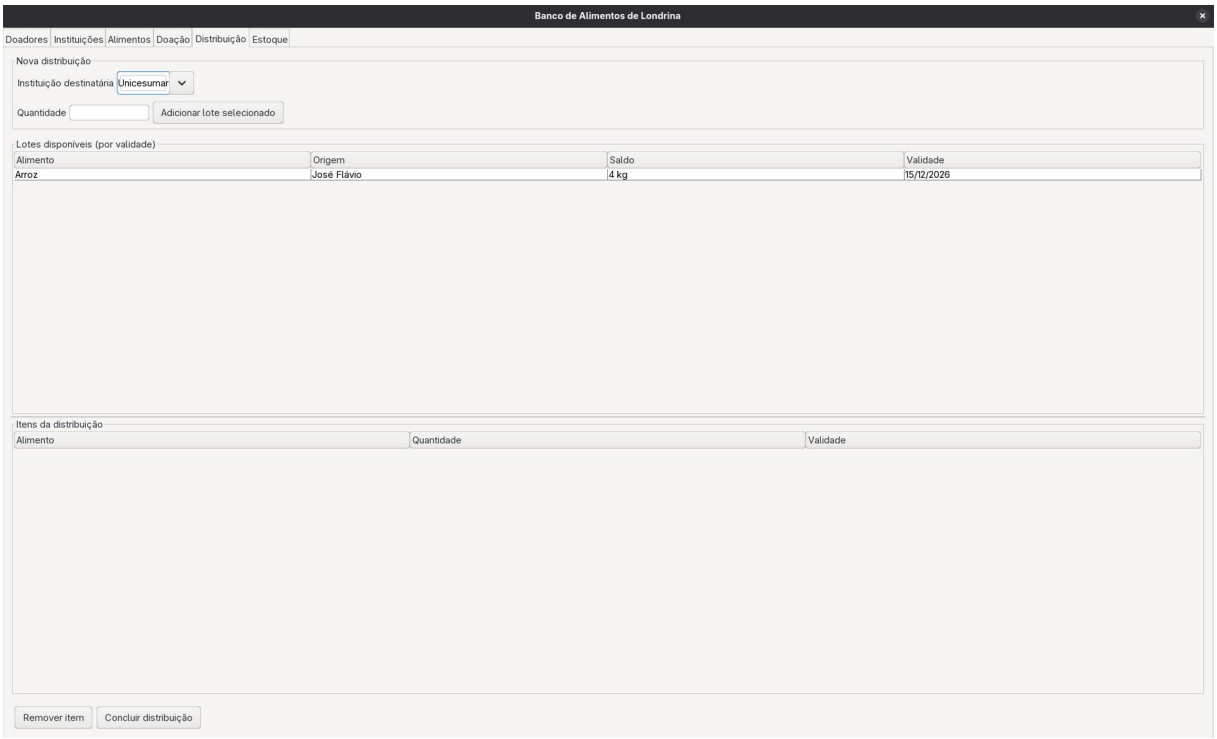
Fonte: Captura de tela do sistema (2026).

Figura 21 – Aba Doação



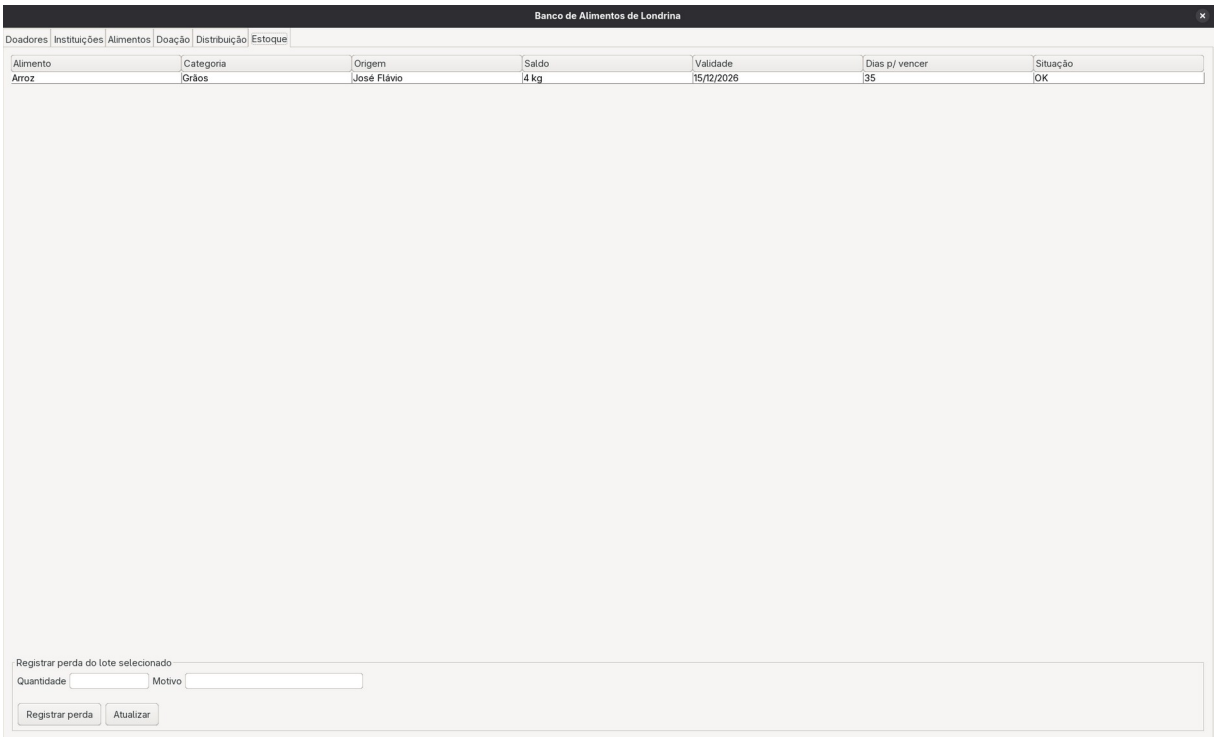
Fonte: Captura de tela do sistema (2026).

Figura 22 – Aba Distribuição



Fonte: Captura de tela do sistema (2026).

Figura 23 – Aba Estoque



Fonte: Captura de tela do sistema (2026).

2.3.11 Fluxo CRUD completo

O CRUD está completo para Doador, Instituicao e Alimento. Em cada um, os serviços fazem o cadastro, a consulta, a listagem, a atualização e a exclusão. O cadastro recusa dados obrigatórios vazios e valores repetidos. A atualização recusada mantém os dados anteriores. A exclusão exige confirmação e é bloqueada quando o registro já foi usado em lotes ou distribuições. Como o histórico de estoque depende desses registros, a exclusão fica bloqueada para preservar a rastreabilidade da origem e do destino dos alimentos.

O fluxo de Doacao também está completo: o usuário cria a doação, adiciona itens e a registra, o que gera os lotes. A consulta é feita na aba Estoque.

2.3.12 Testes

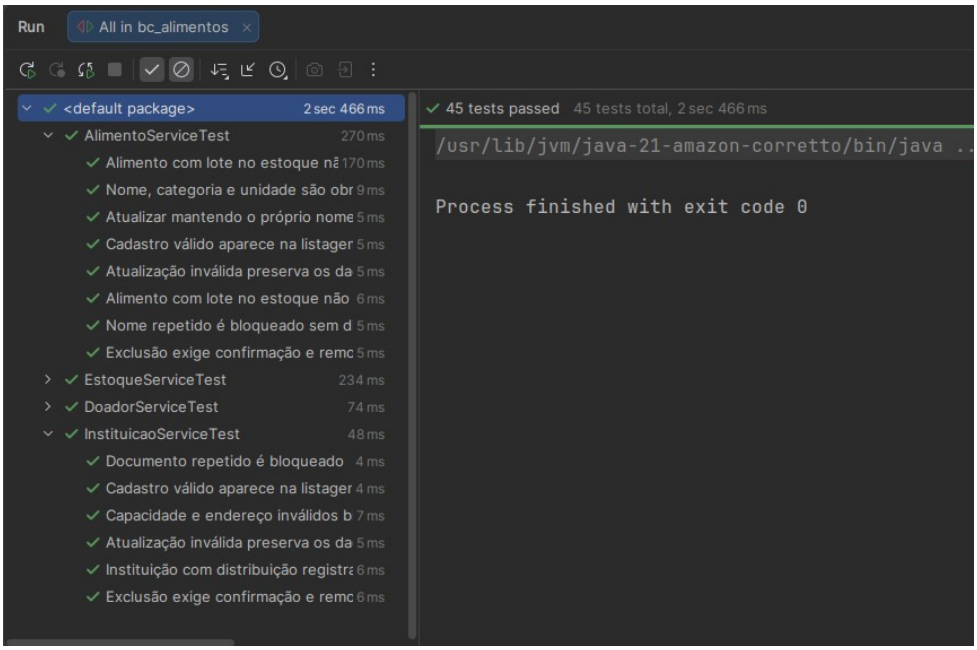
Os testes foram feitos com JUnit 5 e ficam na pasta test. São 45 testes em quatro classes (DoadorServiceTest, InstituicaoServiceTest, AlimentoServiceTest e EstoqueServiceTest), e todos foram aprovados na execução. O Quadro 19 resume os cenários cobertos e a Figura 24 mostra a execução.

Quadro 19 – Cenários de teste

Tipo de cenário	Exemplos testados
Válidos	Cadastro, atualização e exclusão de doador, instituição e alimento; doação que cria lote com origem, validade e saldo; distribuição de 4 de um lote de 10 deixando saldo 6; perda que reduz o saldo
Inválidos	Campos obrigatórios vazios; documento ou nome repetido; quantidade zero, negativa ou nula; atualização inválida que preserva os dados; exclusão sem confirmação; exclusão de registro com lote ou distribuição; troca de unidade de um alimento que já tem lote
Estoque insuficiente	Pedido de 11 com saldo 10; dois itens do mesmo lote que somados passam do saldo; falha em um item sem baixar nenhum dos outros; perda maior que o saldo
Lote vencido	Saída em data posterior à validade; doação com validade anterior à data da doação; lote que vence no próprio dia ainda pode sair; lotes vencidos e esgotados fora da lista de disponíveis

Fonte: Elaborado para este trabalho (2026).

Figura 24 – Execução dos testes JUnit



Fonte: Captura de tela do IntelliJ IDEA (2026).

2.3.13 Documentação e código-fonte

O código-fonte está no repositório do projeto: https://github.com/JoseFlavioXavier/AEP_ENTREGA_02. A organização é a seguinte:

- src: 38 classes Java do sistema (domínio, serviços, repositório e telas).
- test: classes de teste JUnit 5.
- docs/diagramas: imagens dos diagramas de classes e os arquivos de origem (.puml).
- banco: scripts SQL do banco de dados (documentados na seção 2.1).
- sist_op: simulador de sistemas operacionais (documentado na seção 2.2).

Para executar a interface gráfica, basta rodar a classe App. A classe Main faz uma demonstração no console com os mesmos serviços. Os testes rodam pela pasta test no IntelliJ, com a biblioteca JUnit 5 adicionada ao projeto.

Limitação conhecida: os dados ficam apenas em memória e são perdidos ao fechar o programa. O banco de dados modelado na disciplina de Banco de Dados é entregue como scripts SQL e não está ligado ao sistema Java nesta entrega. Como o acesso aos dados passa pela interface CrudRepository, uma implementação com banco poderia ser acrescentada sem alterar as telas nem as regras de negócio.

2.4 ENGENHARIA DE REQUISITOS

2.4.1 Contexto, objetivo e escopo

O Banco de Alimentos de Londrina é uma entidade fictícia que recebe doações de mercados, atacadistas e produtores e as distribui a creches, casas de acolhimento e cozinhas solidárias. O controle por planilhas dificulta rastrear origem, validade e destino dos alimentos, planejar coletas e entregas, calcular indicadores e prestar contas.

O MVP deve digitalizar o ciclo de doações, estoque e distribuição em Java, com banco relacional e separação entre apresentação, regras de negócio e persistência. Esta Especificação de Requisitos de Software (ERS) consolida os requisitos, casos de uso, priorização, rastreabilidade, validação e controle de mudanças. Os modelos de banco, scripts e código pertencem às seções das demais disciplinas.

Dentro do MVP: acesso por perfis; cadastros de doadores, instituições e alimentos; recebimento e triagem; lotes e validade; estoque, reservas, expedições e perdas; ordens de coleta e entrega; relatórios, CSV e auditoria.

Fora do MVP: portal público de doadores, otimização automática de rotas, GPS, integrações fiscais, conversão de unidades e devoluções após expedição. A logística básica permanece no escopo.

2.4.2 Partes interessadas e levantamento

Quadro 20 – Partes interessadas e necessidades

Parte interessada	Necessidade principal
Coordenação	Aprovar escopo e regras; consultar indicadores e auditoria.
Recepção e qualidade	Conferir origem, quantidades, validade, rejeições e perdas.
Distribuição e assistência social	Manter instituições, reservar alimentos e registrar destinos.
Logística	Agendar e acompanhar coletas e entregas.
Doadores e instituições	Obter prestação de contas e confirmar recebimento, por intermédio da equipe.
Equipe e docente	Implementar, testar e avaliar a consistência da solução.

Fonte: Matriz de partes interessadas do grupo (2026).

O cenário considera um galpão com unidades fixas kg, L e un, sem somar unidades diferentes.

2.4.3 Requisitos funcionais e priorização (MoSCoW)

A priorização usa a técnica MoSCoW: M (Must) é obrigatório, S (Should) é importante, C (Could) é opcional e W (Won’t) fica fora desta versão. A prioridade considera valor, risco da ausência e dependências.

Quadro 21 – Requisitos funcionais e prioridade

ID	Especificação resumida	Prior.
RF-001	Registrar doador, data, itens e triagem; somente a quantidade aceita gera entrada.	M
RF-002	Classificar alimentos em perecíveis ou não perecíveis e permitir filtros.	M
RF-003	Manter nome, categoria, unidade e situação dos alimentos; preservar histórico.	M
RF-004	Controlar validade; alertar entre hoje e sete dias; separar lotes vencidos.	M
RF-005	Identificar cada lote por recebimento, alimento, doador, validade e quantidade.	M
RF-006	Exibir agenda de coletas com previsão, realização e pendências atrasadas.	M
RF-007	Exibir agenda de entregas com previsão, realização e pendências atrasadas.	M
RF-008	Receber intenção de doação por link externo, sem criar estoque automaticamente.	W
RF-009	Otimizar rotas considerando paradas e restrições; adiado para versão futura.	W
RF-010	Expedir uma distribuição em transação única, consumindo reservas e baixando saldo.	M
RF-011	Consultar a origem de um lote, seu recebimento e movimentos.	M
RF-012	Consultar instituições destinatárias e quantidades expedidas de cada lote.	M
RF-013	Manter doadores, documentos, contatos, endereços e situação ativa.	M
RF-014	Manter instituições, CNPJ, contato, endereço, tipo de atendimento e situação.	M
RF-015	Consultar saldo físico, reservado e disponível por lote e seus filtros.	M
RF-016	Autenticar usuários e autorizar operações por perfil na interface e no serviço.	M
RF-017	Manter ordens de coleta e entrega, responsável, endereço, datas e ocorrências.	M
RF-018	Reservar alimentos por validade e cancelar reservas antes da expedição.	M
RF-019	Bloquear lotes e registrar perdas por quantidade, motivo, data e responsável.	M
RF-020	Calcular entradas, distribuição, perdas e giro por período, categoria e unidade.	M
RF-021	Exportar CSV de prestação de contas com os filtros e totais da consulta.	M
RF-022	Permitir à coordenação consultar eventos de auditoria por usuário, objeto e data.	M
RF-023	Permitir configurar a janela de alerta de 1 a 30 dias; padrão de sete dias.	S
RF-024	Apresentar gráficos com os mesmos valores dos relatórios.	C

Fonte: ERS do grupo (2026).

Justificativa. Os Must garantem o ciclo operacional e a integridade da prestação de contas. O RF-023 melhora a adaptação, mas o prazo fixo atende ao núcleo; o RF-024 é dispensável porque os relatórios já mostram os valores. Os RF-008 e RF-009 exigem integrações e ampliam a complexidade.

Dependências e ordem. Acesso e cadastros, depois recebimento e lotes, depois estoque, reservas, perdas e expedição, e por fim logística e relatórios. Os RF-006 e RF-007 dependem do RF-017; o RF-010 depende do RF-018; o RF-020 depende dos movimentos; os RF-021 e RF-024 dependem do RF-020. A auditoria deve existir desde as primeiras gravações. Em falta de capacidade, corta-se primeiro C e depois S; lacunas Must devem ser declaradas.

2.4.4 Requisitos não funcionais

Quadro 22 – Requisitos não funcionais

RNF	Meta e verificação proposta
RNF-001 Integridade	Transações atômicas, referências válidas e saldo não negativo; falha induzida deve desfazer toda a operação.
RNF-002 Concorrência	Duas reservas simultâneas de 40 kg sobre saldo 50 permitem só uma. Repetir confirmação não duplica movimento.
RNF-003 Desempenho	Com 4 vCPU, 8 GB, SSD, 100 mil movimentos e 10 sessões: p95 até 2 s em consultas e gravações e 5 s em relatórios. Medir por 10 min após 2 min de aquecimento.
RNF-004 Segurança	Permissões verificadas no serviço; senha com hash adaptativo e salt; conexão cifrada entre máquinas.
RNF-005 Auditoria	Registrar autor, instante, ação e antes/depois; operadores não editam eventos; nunca registrar credenciais.
RNF-006 Recuperação	Backup diário; restauração em até 2 h e perda máxima de 24 h de dados, comprovadas em base isolada.
RNF-007 Usabilidade	Três avaliadores concluem um recebimento em até 3 min após orientação de até 10 min; navegação por teclado.
RNF-008 Testabilidade	Separar camadas e testar automaticamente regras de saldo, validade e autorização.
RNF-009 Precisão	Até três casas decimais em kg/L e somente inteiros em un; datas operacionais no fuso local.
RNF-010 Falhas	Erro de gravação preserva o formulário, não expõe SQL e permite repetição segura com o mesmo ID.

Fonte: ERS do grupo (2026).

O RNF-007 é Should; os demais são Must.

2.4.5 Regras de negócio

Quadro 23 – Regras de negócio da ERS

Código	Regra
RN-001	Novos recebimentos e reservas exigem parceiros ativos; revalidar a instituição na expedição.
RN-002	Documento normalizado único e válido; lote com uma origem, alimento e validade; unidade imutável após movimentação.

Código	Regra
RN-003	Físico = entradas aceitas – saídas – perdas; disponível = físico – reservas ativas para lote elegível, e zero para bloqueado ou vencido.
RN-004	Validade obrigatória, não anterior ao recebimento nem à expedição e entrega prevista; o lote vence quando hoje ultrapassa a data de validade.
RN-005	Usar FEFO (primeiro que vence, primeiro que sai); empate por recebimento e ID; exceção exige motivo.
RN-006	Recebida = aceita + rejeitada; quantidades não negativas, recebida maior que zero e rejeição justificada. Rejeitados não compõem estoque nem perdas de estoque.
RN-007	Confirmação integral e única; nenhuma reserva supera o físico.
RN-008	A reserva reduz o disponível; o cancelamento anterior à expedição libera saldo.
RN-009	Bloquear ou registrar perda cancela as reservas do lote e exige replanejamento; a perda não supera o físico.
RN-010	O vencimento invalida reservas, mantendo o físico até a baixa por perda.
RN-011	A ordem passa de agendada a em execução e concluída; a entrega inicia após a expedição; a parcialidade gera ocorrência e nova ordem, sem nova baixa.
RN-012	A distribuição passa de reservada a expedida e entregue; a conclusão exige recebedor e data.
RN-013	Movimentos confirmados não são editados ou excluídos.
RN-014	Alertas de validade usam sete dias e a agenda destaca ordens abertas de hoje ou atrasadas.
RN-015	Separar unidades e considerar movimentos confirmados no período inclusivo. Perdas % = perdas ÷ (estoque inicial + entradas) × 100; giro = distribuído ÷ média dos saldos físicos ao fim de cada dia. Denominador zero: “não aplicável”.
RN-016	Contas individuais; só a coordenação administra usuários e parâmetros. Cada objeto mantém ID; movimentos vinculam lote, quantidade, tipo, autor, instante e operação de origem.

Fonte: ERS do grupo (2026).

2.4.6 Casos de uso

Os casos de uso seguem a ordem dos identificadores. Cada um traz ator e pré-condições, fluxo principal, alternativas e exceções, e pós-condição com os critérios de aceite, que se referem aos casos de teste CT-01 a CT-24.

UC-01 Acesso

Ator e pré-condições. Usuário e coordenação.

Fluxo principal. Informar credenciais, validar conta ativa e abrir funções do perfil. Sair encerra a sessão; a coordenação pode redefinir senha.

Alternativas e exceções. Conta inativa ou operação sem permissão é negada.

Pós-condição e aceite. CT-01: consulta não grava recebimento.

UC-02 Cadastros

Ator e pré-condições. Recepção ou distribuição.

Fluxo principal. Preencher dados obrigatórios e validar o documento; salvar com auditoria.

Alternativas e exceções. Edição e inativação preservam histórico. Duplicidade e troca de unidade já movimentada são recusadas.

Pós-condição e aceite. CT-02/03: documento único e unidade imutável.

UC-03 Receber e triar doação

Ator e pré-condições. Recepção; doador e alimento ativos; quantidades conferidas e ID único da operação.

Fluxo principal. 1. Selecionar doador e coleta, se houver. 2. Informar data, alimento, validade e quantidades recebida, aceita e rejeitada. 3. Justificar rejeições. 4. Conferir o resumo. 5. Confirmar recebimento, lotes e entradas na mesma transação.

Alternativas e exceções. Doação espontânea dispensa coleta. Rejeição integral não gera lote. Validades diferentes geram lotes diferentes. Soma incorreta, validade inválida ou campo ausente impedem a confirmação; falha técnica desfaz a operação.

Pós-condição e aceite. O estoque recebe somente o aceito. CT-04: 100 kg recebidos, 90 aceitos e 10 rejeitados geram 90 kg de estoque. CT-05: repetir o mesmo ID conserva uma entrada. CT-06: duas validades geram dois lotes rastreáveis.

UC-04 Estoque

Ator e pré-condições. Operador autorizado.

Fluxo principal. Filtrar lotes, avaliar validade e mostrar três saldos; abrir a origem.

Alternativas e exceções. Lista vazia é permitida; falha não apresenta saldo antigo como atual.

Pós-condição e aceite. CT-07/08: janela de sete dias; físico 100 menos reserva 30 resulta em 70 livres.

UC-05 Reservar ou cancelar distribuição

Ator e pré-condições. Distribuição; instituição ativa, entrega prevista e lotes elegíveis.

Fluxo principal. 1. Selecionar instituição e quantidades. 2. O sistema sugere lotes por FEFO. 3. O operador confere e justifica eventual troca. 4. O sistema revalida o saldo na transação e grava reservas sem baixar o físico.

Alternativas e exceções. Dividir quantidades entre lotes; cancelar antes da expedição libera reservas. Replanejar libera reservas remanescentes e cria nova reserva. Saldo insuficiente, concorrência, instituição inativa ou validade anterior à entrega impedem a confirmação.

Pós-condição e aceite. Distribuição reservada ou cancelada. CT-09: duas reservas simultâneas de 40 kg sobre 50 kg permitem só uma. CT-10: cancelamento recompõe o disponível. CT-11: sugerir o lote elegível com menor validade.

UC-06 Confirmar expedição

Ator e pré-condições. Distribuição; reserva válida, instituição ativa e ordem de entrega agendada.

Fluxo principal. 1. Conferir lotes separados. 2. Revalidar saldo, reserva, qualidade e validade em relação à entrega. 3. Confirmar com ID único. 4. Consumir reservas e registrar saídas integralmente, vinculadas à instituição.

Alternativas e exceções. Divergência exige cancelar e reservar novamente antes da saída. Repetição retorna o resultado existente. Lote bloqueado, vencido ou falha em qualquer item impede toda a expedição.

Pós-condição e aceite. Distribuição expedida, com destino rastreável. CT-12: físico 100 e reserva 30 passam a físico 70 e reserva zero; nova confirmação não baixa novamente. CT-13: falha no segundo lote deixa todos os saldos inalterados.

UC-07 Bloquear lote ou registrar perda

Ator e fluxo. Qualidade; localizar lote existente, informar ação e motivo, indicar quantidade na perda, conferir reservas afetadas e confirmar.

Alternativas e exceções. Desbloqueio justificado exige lote não vencido e não reativa reservas. Perda parcial é permitida; quantidade maior que o físico ou não positiva é recusada.

Resultado e aceite. Reservas afetadas são canceladas e distribuições exigem replanejamento. CT-14: perda de 5 kg sobre saldo 20 deixa 15. CT-15: lote bloqueado não pode ser expedido.

UC-08 Gerenciar coletas e entregas

Ator e pré-condições. Logística; parceiro cadastrado; entrega vinculada a uma distribuição.

Fluxo principal. 1. Informar tipo de ordem, parceiro, endereço, previsão e responsável. 2. Agendar. 3. Iniciar a execução; a entrega exige expedição anterior. 4. Registrar ocorrência e data efetiva; a entrega exige recebedor. 5. Concluir e atualizar a situação.

Alternativas e exceções. Reagendar com motivo. Cancelar entrega ainda agendada libera a distribuição reservada. Entrega parcial mantém a distribuição expedida e cria ordem para o restante, sem repetir a baixa. Campos ausentes ou transições inválidas são recusados.

Pós-condição e aceite. Ordem atualizada com histórico. CT-16: coleta concluída não cria estoque antes da conferência. CT-17: concluir entrega não gera segunda saída. CT-18: ordem vencida e aberta aparece como atrasada.

UC-09 Consultar relatórios e exportar

Ator e pré-condições. Coordenação ou consulta; acesso autorizado aos movimentos.

Fluxo principal. 1. Selecionar período, categoria, unidade e parceiro. 2. Calcular indicadores dos movimentos confirmados. 3. Consultar o detalhamento por lote. 4. Exportar CSV com os mesmos filtros.

Alternativas e exceções. Sem movimentos, volumes são zero; quociente sem denominador é “não aplicável”. Datas invertidas são recusadas; unidades distintas são apresentadas separadamente. Falha de exportação permite nova tentativa sem alterar o estoque.

Pós-condição e aceite. Relatório consultado ou exportado. CT-19: inicial 100 kg, entradas 50, saídas 40 e perdas 10 resultam em final 100 e perda de 6,67%. CT-20: CSV confere com a tela. CT-21: saldos diários de 100, 80 e 60 e distribuição 40 geram giro de 0,5 vez.

UC-10 Auditoria

Ator e pré-condições. Coordenação.

Fluxo principal. Filtrar eventos e consultar autor, instante e antes/depois.

Alternativas e exceções. Sem resultados, informar os filtros; alteração de evento é vedada.

Pós-condição e aceite. CT-22: mudança de contato é rastreável, sem exposição de credenciais.

UC-11 Parâmetro

Ator e pré-condições. Coordenação.

Fluxo principal. Se implementado, informar e salvar a janela de 1 a 30 dias; cancelar preserva o valor.

Alternativas e exceções. Fora do intervalo é recusado.

Pós-condição e aceite. CT-23: mudar sete para dez inclui lote que vence em nove dias.

UC-12 Painel

Ator e pré-condições. Consulta autorizada.

Fluxo principal. Se implementado, filtrar e visualizar gráficos com detalhamento.

Alternativas e exceções. Sem dados, informar ausência; filtro inválido é recusado.

Pós-condição e aceite. CT-24: gráfico reproduz os valores do relatório.

2.4.7 Rastreabilidade

O Quadro 24 relaciona requisitos, regras de negócio, casos de uso, funcionalidades e casos de teste. O Quadro 25 informa a situação de cada requisito funcional no MVP efetivamente construído, e o Quadro 26 informa como cada caso de teste foi verificado (UNICESUMAR, 2026, p. 7).

Quadro 24 – Matriz de rastreabilidade

Requisitos	Regras	Casos	Funcionalidade e teste
RF-016	RN-016	UC-01	F01 Acesso • CT-01
RF-002/003/013/014	RN-001/002	UC-02	F02 Cadastros • CT-02/03
RF-001/005	RN-001/002/006/007	UC-03	F03 Recebimento • CT-04/05/06
RF-004/011/015	RN-003/004/010/014	UC-03/04	F04 Estoque e origem • CT-07/08
RF-018	RN-003/005/007/008	UC-05	F05 Reserva • CT-09/10/11
RF-010/012	RN-003/004/007/012	UC-06	F06 Saída e destino • CT-12/13
RF-019	RN-003/009/010	UC-07	F07 Qualidade • CT-14/15
RF-006/007/017	RN-011/012/014	UC-08	F08 Logística • CT-16/17/18
RF-020/021	RN-015/016	UC-09	F09 Relatórios e CSV • CT-19/20/21
RF-022	RN-013/016	UC-10	F10 Auditoria • CT-22
RF-023/024	RN-014/015/016	UC-11/12	F11 Parâmetros e F12 Painel • CT-23/24
RF-008/009	A refinar	Futuros	Portal e rotas • fora do MVP

Fonte: Matriz do grupo (2026), com a numeração das funcionalidades (F) e dos casos de teste (CT) ajustada para evitar repetições.

RNF-001/002/010: testar transações e repetição em UC-03/05/06/07. RNF-003: medir carga em estoque, gravações e relatórios. RNF-004/005: verificar permissões e auditoria. RNF-006: restaurar backup. RNF-007: observar tarefas na interface. RNF-008: revisar camadas e testes. RNF-009: testar unidades, decimais e virada de data.

Quadro 25 – Situação dos requisitos funcionais no MVP construído

RF	Situação	Onde está e o que falta
RF-001	Parcial	A doação com itens gera lotes (classe Doacao, aba Doação, procedure sp_registrar_doacao). Não há etapa de triagem com quantidade rejeitada.
RF-002	Não implementado	Há categoria do alimento, mas não o indicador perecível ou não perecível.
RF-003	Parcial	CRUD de alimento com nome, categoria e unidade de medida; exclusão bloqueada quando há lotes. Falta a situação do alimento.
RF-004	Implementado	Aba Estoque (situação “vence em até 7 dias”, vencido, esgotado), função fn_situacao_lote e view vw_estoque_validade. Lotes vencidos ficam fora da lista de distribuição.
RF-005	Implementado	Entidade Lote (alimento, doador de origem, validade, saldo) e tabelas doacao e lote.
RF-006	Não implementado	Sem agenda de coletas.
RF-007	Não implementado	Sem agenda de entregas.
RF-008	Fora do MVP	Prioridade W.
RF-009	Fora do MVP	Prioridade W.

RF	Situação	Onde está e o que falta
RF-010	Parcial	A distribuição é atômica e baixa o saldo (Distribuicao.concluir e sp_registrar_distribuicao). Não há reservas.
RF-011	Parcial	A origem aparece na aba Estoque; o histórico de movimentos está na view vw_rastreabilidade_lote.
RF-012	Parcial	Os destinos e as quantidades por lote estão na view vw_rastreabilidade_lote e na consulta 11; a aplicação Java não tem essa consulta.
RF-013	Parcial	CRUD de doador com documento, contato e tipo. Faltam endereço e situação ativa.
RF-014	Parcial	CRUD de instituição com documento, contato, endereço e capacidade. Faltam tipo de atendimento e situação.
RF-015	Parcial	Saldo físico por lote na aplicação e no banco. Não há saldo reservado.
RF-016	Não implementado	Sem login nem perfis.
RF-017	Não implementado	Sem ordens de coleta e entrega.
RF-018	Não implementado	Sem reservas.
RF-019	Parcial	Perda com quantidade, motivo e data (classe Perda, aba Estoque, sp_registrar_perda). Faltam o bloqueio de lote e o responsável.
RF-020	Parcial	Views vw_doacoes_periodo e vw_indicadores_categoria e consultas 7 e 8, por período, categoria e unidade. As fórmulas diferem da RN-015 e não há tela na aplicação.
RF-021	Não implementado	Sem exportação CSV.
RF-022	Não implementado	Sem auditoria de eventos.
RF-023	Não implementado	A janela de alerta é fixa em sete dias.
RF-024	Não implementado	Sem gráficos.

Fonte: Elaborado para este trabalho (2026).

Resumo: 2 requisitos implementados, 10 parciais, 10 não implementados e 2 fora do MVP, de um total de 24.

Quadro 26 – Verificação dos casos de teste

CT	Situação	Como foi verificado ou o que falta
CT-01	Não implementado	Sem login; não se aplica ao MVP construído.
CT-02	Implementado	Teste JUnit de documento repetido bloqueado (DoadorServiceTest) e cenário 02 do banco.
CT-03	Implementado	Teste JUnit de troca de unidade com lote (AlimentoServiceTest) e cenário 21 do banco.
CT-04	Não implementado	Não há quantidade rejeitada; a doação registra apenas a quantidade informada.

CT	Situação	Como foi verificado ou o que falta
CT-05	Não implementado	Sem identificador de operação para repetição segura.
CT-06	Parcial	Cada item de doação cria um lote; os dados de teste do banco têm doações com mais de um lote.
CT-07	Implementado	Aba Estoque e consulta 4 do banco mostram os lotes que vencem em até 7 dias.
CT-08	Não implementado	Sem reservas.
CT-09	Não implementado	Sem reservas.
CT-10	Não implementado	Sem reservas.
CT-11	Parcial	A lista de lotes disponíveis é ordenada por validade; não há sugestão automática.
CT-12	Parcial	Teste JUnit de distribuição de 4 de um lote de 10 deixando saldo 6; sem reserva e sem confirmação repetida.
CT-13	Implementado	Teste JUnit de falha em um item sem baixar nenhum dos outros e cenário 20 do banco.
CT-14	Implementado	Teste JUnit de perda que reduz o saldo e cenários 13 e 14 do banco.
CT-15	Parcial	Lote vencido ou esgotado não sai (testes JUnit e cenário 10); não há bloqueio manual de lote.
CT-16	Não implementado	Sem ordens de coleta.
CT-17	Não implementado	Sem ordens de entrega.
CT-18	Não implementado	Sem ordens de coleta e entrega.
CT-19	Parcial	O banco calcula a perda sobre a quantidade doada (consulta 7); a fórmula da RN-015 não foi aplicada.
CT-20	Não implementado	Sem exportação CSV.
CT-21	Parcial	O banco calcula o giro como distribuído sobre doado; a fórmula da RN-015 não foi aplicada.
CT-22	Não implementado	Sem auditoria.
CT-23	Não implementado	Sem parâmetro configurável.
CT-24	Não implementado	Sem gráficos.

Fonte: Elaborado para este trabalho (2026), a partir dos testes JUnit (45) e dos cenários do banco (21).

2.4.8 Divergências entre a ERS e a implementação

A ERS descreve o MVP completo e a implementação cobre parte dele. Além do que mostra o Quadro 25, há quatro diferenças a registrar, que não alteram as regras da ERS:

- Unidades: a ERS fixa kg, L e un; a aplicação e o banco aceitam kg, g, l, ml e un, também sem conversão entre elas.
- Indicadores: a RN-015 define a perda sobre estoque inicial mais entradas e o giro sobre a média dos saldos diários; o banco calcula ambos sobre a quantidade doada.

- Unidade “un”: o RNF-009 exige quantidades inteiras, e essa validação não existe no banco nem na aplicação.
- Códigos: os requisitos, regras e casos de uso da seção de POO têm o prefixo POO- (por exemplo, POO-RF01), para não se confundirem com os desta seção.

2.4.9 Protótipo e validação dos requisitos

A Figura 25 é um protótipo de baixa fidelidade para os casos UC-05 e UC-06, com dados fictícios. A interface distingue físico, reservado e disponível, permite conferir a origem, cancelar a reserva ou expedir. Antes da confirmação, o serviço revalida validade, saldo e situação da instituição. No recebimento, os campos obrigatórios são doador, data, alimento, validade e quantidades, e o resumo deve informar a entrada exata antes da gravação. As telas reais do sistema construído estão na seção de POO.

Figura 25 – Protótipo de baixa fidelidade da distribuição (UC-05 e UC-06)



Fonte: Protótipo do grupo (2026), com dados fictícios.

A revisão documental corrigiu a prioridade alta generalizada, separou lote de entrada e instituição destinatária, distinguiu previsão e realização logística e substituiu o aceite do RF-010 por comprovação de baixa. A expressão “próximo da validade” passou a significar sete dias. Essa revisão não equivale à homologação do software.

Os casos CT-01 a CT-24 devem ser executados com parceiros fictícios e contas de cada perfil; os CT-23 e CT-24 dependem dos opcionais. Para a validade, fixa-se 05/11/2026: o lote de 04/11 está vencido, o de 12/11 entra no alerta e o de 13/11 fica fora. A massa é reinicializada a cada teste. Para a rastreabilidade, distribui-se um lote a duas instituições e conferem-se ambas as quantidades e a origem.

Ensaios técnicos. Simular falha no meio da gravação, reservas concorrentes, reenvio do mesmo ID e tentativa de acesso indevido; medir desempenho, restaurar backup e comparar saldos; verificar decimais, auditoria, separação de camadas e uso por teclado conforme os RNFs.

Registro de teste. Guardar ID, versão da ERS e do software, ambiente, massa, executor, data, resultado esperado, resultado obtido, evidência, situação e reteste. Estado atual: os testes automáticos do que foi construído foram executados (45 testes JUnit e 21 cenários do banco, Quadro 7); os testes operacionais e o aceite com recepção, qualidade, distribuição e logística continuam pendentes.

Aceite. Todos os Must atendidos e testes obrigatórios aprovados; nenhuma falha que permita saldo negativo, saída duplicada, lote inelegível, acesso indevido ou quebra de rastreabilidade. Registrar responsáveis, data e ressalvas. Valores e evidências não devem ser apresentados como resultados executados antes da realização dos testes.

2.4.10 Gestão de mudanças

O fluxo é: proposta, análise, aprovação ou rejeição, implementação e verificação. Cada solicitação registra ID, requisito, texto anterior e novo, justificativa, impacto em dados, código e testes, responsável, decisão e versão. A coordenação aprova mudanças de escopo e a equipe registra ajustes editoriais. Aprovação não significa implementação, e os IDs não são reutilizados. O Quadro 27 compara a versão 1.0 (matriz da 1ª entrega) com a 2.0 e registra os ajustes desta consolidação (2.1).

Quadro 27 – Registro de mudanças

ID	Vers.	Requisito	Antes e depois	Justificativa	Impacto
PM-001	2.0	RF-005, RF-012	Antes: o lote era criado com a entidade receptora. Depois: o lote identifica recebimento, alimento, doador, validade e quantidade, e o destino fica na saída.	O destino só existe na saída do lote.	RF-005/012, UC-03/06 e seus testes.
PM-002	2.0	RF-006, RF-007, RF-017	Antes: informar data de chegada e de saída com alertas. Depois: agendas com previsão, realização e pendências atrasadas.	Distinguir o planejado do realizado.	Agenda e UC-08.
PM-003	2.0	RF-010	Antes: retirada atualizando o estoque. Depois: expedição em transação única, com aceite por comprovação de baixa.	Garantir baixa única.	Transação, UC-06 e CT-12/13.
PM-004	2.0	RF-004, RF-023	Antes: alerta para produtos próximos da validade, sem prazo. Depois: janela de sete dias, configurável de 1 a 30 dias (opcional).	Tornar o alerta testável.	Alertas e testes de limite.

ID	Vers.	Requisito	Antes e depois	Justificativa	Impacto
PM-005	2.0	RF-008, RF-009	Antes: prioridade alta. Depois: fora do MVP (W), mantendo a logística básica.	Exigem integrações e ampliam a complexidade.	Portal e rotas adiados.
PM-006/007	2.0	RF-013 a RF-024, RNF, MoSCoW	Antes: 12 requisitos, todos de prioridade alta. Depois: 24 requisitos com MoSCoW, 10 RNFs e 16 regras.	Cobrir cadastros, reservas, perdas, relatórios e qualidade.	Documento inteiro.
PM-008	2.1	Matriz de rastreabilidade	Antes: sem a situação de implementação. Depois: Quadros 25 e 26 com a situação de cada RF e de cada CT.	O enunciado pede rastreabilidade com as funcionalidades implementadas (UNICESUMAR, 2026, p. 7).	Quadros 25 e 26.
PM-009	2.1	RN-002, RN-015, RNF-009	Antes: sem registro das diferenças. Depois: divergências entre a ERS e a implementação documentadas, sem alterar as regras.	Mostrar com clareza o que foi entregue.	Seção 2.4.8.

Fonte: Registro de mudanças do grupo (2026).

3 CONSIDERAÇÕES FINAIS

O trabalho entrega um MVP do Banco de Alimentos de Londrina em quatro frentes que se completam: um banco de dados MariaDB com tabelas, restrições, triggers, procedures e views, com 21 cenários de erro bloqueados; um simulador de sistemas operacionais com sincronização por mutex e semáforos e a comparação entre FIFO e LRU; um sistema Java com interface gráfica, CRUD de doadores, instituições e alimentos, doações, distribuições e perdas, com 45 testes automatizados aprovados; e uma ERS com 24 requisitos, 12 casos de uso, rastreabilidade e gestão de mudanças.

Algumas limitações ficam registradas. A aplicação Java usa persistência em memória e não está ligada ao banco de dados; como o acesso passa pela interface CrudRepository, uma implementação com JDBC pode ser acrescentada sem alterar as telas nem as regras de negócio. A ERS descreve um escopo maior que o construído, e o Quadro 25 mostra a situação de cada requisito. O simulador de sistemas operacionais usa contadores próprios, sem ligação com o estoque da aplicação, e a comparação entre FIFO e LRU terminou em empate com a sequência utilizada.

Como próximos passos, destacam-se a integração entre a aplicação Java e o banco, a implementação de login e perfis, de reservas de estoque e de ordens de coleta e entrega, e uma nova rodada de testes do cache com sequências de acessos repetidos.

REFERÊNCIAS

GEEKSFORGEEKS. **Page Replacement Algorithms in Operating Systems**. GeeksforGeeks, 2026a. Disponível em: https://www.geeksforgeeks.org/page-replacement-algorithms-in-operating-systems/. Acesso em: 8 out. 2026.

GEEKSFORGEEKS. **Race Conditions**. GeeksforGeeks, 2026b. Disponível em: https://www.geeksforgeeks.org/java/race-conditions/. Acesso em: 8 out. 2026.

IBM. **Virtual memory page replacement algorithms**. IBM, 2026b. Disponível em: https://www.ibm.com/docs/en/aix/7.3?topic=management-virtual-memory-page-replacement-algorithms. Acesso em: 8 out. 2026.

JIANG, Fontaine. **Critical Section**. Fontaine Jiang's Blog, 17 out. 2017. Disponível em: https://fontainejiang.wordpress.com/2017/10/17/critical-section/. Acesso em: 8 out. 2026.

MICROSOFT. **Objetos semáforos**. Microsoft Learn, 12 mar. 2025. Disponível em: https://learn.microsoft.com/pt-br/windows/win32/sync/semaphore-objects. Acesso em: 11 nov. 2026.

ORACLE. **Semaphore**. Oracle, 2026c. (Java Platform, Standard Edition 26 API Specification). Disponível em: https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/util/concurrent/Semaphore.html. Acesso em: 11 nov. 2026.

UNICESUMAR. **AEP 2026.2**: Engenharia de Software, 4º semestre. [S. l.]: Unicesumar, 2026. 8 p. Enunciado de atividade acadêmica.

BIBLIOGRAFIA CONSULTADA

DATE, C. J. **Introdução a sistemas de bancos de dados**. Tradução de Daniel Vieira. Rio de Janeiro: Elsevier, 2003.

HEUSER, Carlos Alberto. **Projeto de banco de dados**. 6. ed. Porto Alegre: Bookman, 2009.

IBM. **Understanding threads and processes**. IBM, 2026a. Disponível em: https://www.ibm.com/docs/pt-br/aix/7.3.0?topic=programming-understanding-threads-processes. Acesso em: 8 out. 2026.

ISMAEL. **C++ Intro OS Kernel**. Rio de Janeiro: PUC-Rio, [20--]. 1 apresentação de slides (notas de aula). Disponível em: https://web.tecgraf.puc-rio.br/~ismael/Cursos/YC++/aulas/Basic/ModuloI/C++_Intro_OS_Kernel.pdf. Acesso em: 8 out. 2026.

ORACLE. **Memory Management and Caching Strategies**. Oracle, 2026a. (The Java™ Documentation & Tutorials). Disponível em: https://docs.oracle.com/en/java/. Acesso em: 8 out. 2026.

ORACLE. **Processes and Threads**. Oracle, 2026b. (The Java™ Tutorials). Disponível em: https://docs.oracle.com/javase/tutorial/essential/concurrency/procthread.html. Acesso em: 8 out. 2026.

SCHEFFER, Rosely. **Uma visão geral sobre threads**. Campo Digital, [S. l.], v. 2, n. 1, 2007. Disponível em: https://revista2.grupointegrado.br/revista/index.php/campodigital/article/view/311/145. Acesso em: 8 out. 2026.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. **Operating System Concepts**: Page Replacement. 9. ed. Rio de Janeiro: LTC, 2015.

TANENBAUM, Andrew S.; BOS, Herbert. **Sistemas operacionais modernos**. 4. ed. São Paulo: Pearson Education do Brasil, 2016.